Documento de diseño: Proyecto 2, Entrega única

Integrantes

- Juan Nicolás Pérez Ripe — 202522932
- Luis Esteban Acevedo — 202521217
- Juan David Parrado — 202520852

1. Contexto y alcance

1.1 Descripción del problema

Para este proyecto se debe programar el software del Club Deportivo Séneca, la rama deportiva de la universidad. CDS forma a los integrantes del club en diversos deportes por categorías de edad y representa a la universidad en torneos y competencias. La operación se puede segmentar en dos grandes frentes: la gestión deportiva y la comercial (cafetería y tienda deportiva).

El club debe llevar un control de los que juegan, los equipos, las reglas, resultados de partidos, ventas, etc. El sistema que se presenta en este documento maneja esta lógica de negocio y guarda la información necesaria para operar.

Para el Proyecto 2 se agregan torneos internos, recreativos y competitivos, en los que pueden participar socios, empleados, estudiantes de la universidad y personas externas, y una interfaz por consola con un programa para cada tipo de usuario.

Hay 3 principales actores (usuarios) en este sistema: los socios (jugadores), los empleados (entrenadores y fisioterapeutas) y el administrador. Para los torneos se agregan los estudiantes de la universidad y las personas externas, que entran por la misma consola de los socios.

1.2 Objetivos

1. Implementar la lógica del club

2. Verificar las reglas de negocio en el software

3. Ofrecer una consola para cada tipo de usuario, con autenticación y validación de entradas

4. Operar el negocio con el programa persistiendo la información suminstrada

5. Organizar torneos internos y premiar a sus ganadores

1.3 Requeriemientos funcionales

En el documento de análisis se encuentra de forma detallada cada uno de los RFs definidos, y en el documento de historias de usuario del Proyecto 2 están las historias del banco oficial y las de torneos. Aquí se encuentran agrupados por área del negocio a alto nivel.

Área	El sistema debe permitir	ID RF
Acceso	Autenticar a cualquier usuario con login y password, y limitar lo que puede hacer según su rol	RF-S01
Deportes de conjunto	Crear y dar de baja equipos y socios, inscribir jugadores con posición y camiseta, registrar partidos internos con marcador, estadísticas y tarjetas.	RF-S02, RF-EN04, RF-EN05, RF-A08
Competencias	Inscribir un equipo con la nómina, registrar partidos oficiales con rival, escenario y alineación. consultar la tabla de posiciones y las clasificaciones.	RF-EN06, RF-EN07, RF-S10
Academias	Inscribir socios y asignarles nivel, reservar instalaciones, programar partidos de práctica, registrar resultados y actualizar el ranking.	RF-S03 a RF-S07, RF-EN03, RF-EN09
Entrenamiento	Planear sesiones. registrar asistencia y evaluación, consultar las sesiones a las que un socio está convocado.	RF-EN01, RF-EN02, RF-S09
Salud	Registrar y consultar las restricciones médicas de los jugadores.	RF-F01, RF-F02
Tienda y cafetería	Comprar artículos y productos con impuestos, descuentos, propina y puntos, sugerir y aprobar productos nuevos.	RF-S11 a RF-S13, RF-E03, RF-E04, RF-A04
Inventario	Consultar, crear y dar de baja artículos, reabastecer, trasladar unidades entre ubicaciones.	RF-A01 a RF-A03
Turnos	Consultar turnos, solicitar y resolver cambios respetando el mínimo de personal.	RF-E01, RF-E02, RF-A05
Reportes	Desempeño por jugador, ingresos por rubro y por periodo, historial de reservas, partidos y entrenamientos.	RF-S08, RF-EN08, RF-A06, RF-A07
Torneos internos	Crear torneos recreativos y competitivos, inscribir y retirar participantes con cupos preferenciales, registrar resultados, entregar premios y consultar su estado e historial	HU-ADM-04 a HU-ADM-06, HU-SOC-03, HU-SOC-04, HU-EMP-01
Bonos	Guardar el bono ganado en un torneo recreativo y usarlo en una compra futura, sin acumularlo con otros descuentos	HU-SOC-05

1.4 Requeriemientos no funcionales

ID	Requerimiento
RNF-01	Persistencia: que toda la información se conserve entre ejecuciones y que operaciones no se pierdan.
RNF-02	Independencia de la interfaz: que la lógica no depende de cómo se interactúa con el usuario.
RNF-03	Independencia del almacenamiento: que cambiar el formato o la organización de los archivos no obliga a modificar las clases que aplican las reglas.
RNF-04	Verificabilidad: da funcionalidad pide la información necesaria para correr y muestra el resultado de forma clara y fácil de leer.
RNF-05	Control de acceso: toda operación la hace un usuario autenticado y a través de su rol se determina que puede y no puede hacer.

1.5 Restricciones

- La aplicación se escribe en Java y se desarrolla en Eclipse.
- La información se guarda en archivos, dentro de una carpeta distinta a la del código.
- Nunca se escribe informacion de forma manual en los archivos, solo la app.
- La interfaz es por consola, con un programa (main) distinto para cada tipo de usuario.
- Todos los usuarios que ingresan deben tener usuario y password.

1.6 No objetivos

- Interfaz gráfica y dahsboards
- Emparejar automáticamente jugadores según su ranking
- Cifrado de secretos
- Varios usuarios usando el sistema al mismo tiempo
- Generar automáticamente un calendario con fechas y horarios para los partidos
- Gestión de nómina

1.7 Historias de usuario del banco oficial

El grupo implementa las 8 historias del banco oficial, no solo las 3 mínimas. Cada una cumple todos sus criterios de aceptación y tiene su prueba JUnit.

ID	Historia	Implementada

HU-ADM-01	Administrador: trasladar un producto entre tiendas del club	Si
HU-ADM-02	Administrador: revisar solicitudes de cambio de turno	Si
HU-ADM-03	Administrador: aprobar o rechazar sugerencias de nuevos artículos	Si
HU-ENT-01	Entrenador: definir la alineación para un partido oficial	Si
HU-ENT-02	Entrenador: verificar el mínimo de jugadores habilitados	Si
HU-JUG-01	Jugador: reservar una cancha o mesa de deporte individual	Si
HU-SOC-01	Socio: reservar una instalación para práctica individual	Si
HU-SOC-02	Socio: inscribirse en un equipo de conjunto	Si

1.8 Tests

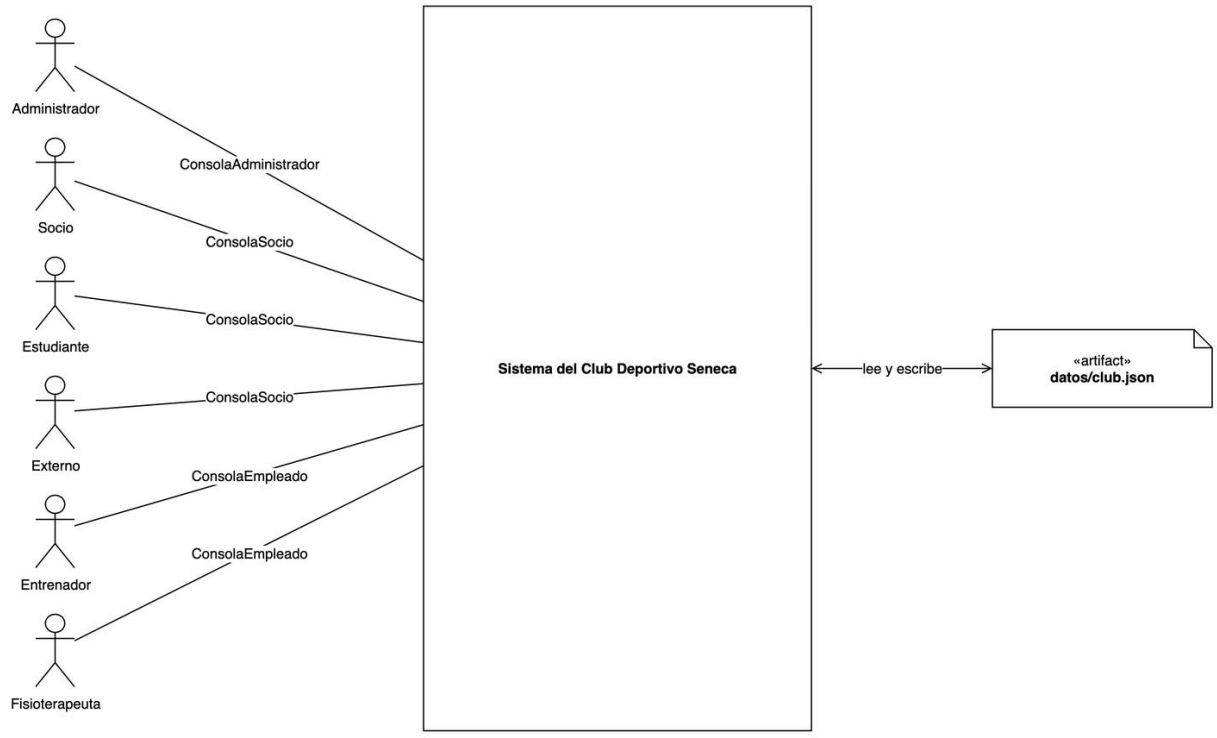
A continuación se muestr auna tabla con cada historia de usuario, del banco y de torneos, con las clases que la implementany la prueba JUnit correspondiente.

Historia	**Clases que la resuelven**	**Prueba JUnit**
HU-ADM-01	TiendaCafeteria, Inventario	HuAdm01TrasladoProductoTest
HU-ADM-02	Club, Turno, SolicitudCambioTurno	HuAdm02CambioTurnoTest
HU-ADM-03	Sugerencia, TiendaCafeteria	HuAdm03SugerenciasTest
HU-ENT-01	Partido, PartidoCompetencia	HuEnt01AlineacionTest
HU-ENT-02	Equipo	HuEnt02MinimoHabilitadosTest
HU-JUG-01	Club, Academia, Reserva	HuJug01ReservaDeporteIndividualTest
HU-SOC-01	Club, Academia, Reserva	HuSoc01ReservaInstalacionTest

HU-SOC-02	Club, Equipo	HuSoc02InscripcionEquipoTest
HU-ADM-04	Club, Torneo, Especialidad	HuAdm04CrearTorneoTest
HU-ADM-05	Club, Estudiante, Externo	HuAdm05InscribirTest
HU-ADM-06	Torneo, EncuentroTorneo, TorneoRecreativo, TorneoCompetitivo	HuAdm06ResultadosTest
HU-SOC-03	Club, RankingTorneos, Torneo, InscripcionTorneo	HuSoc03InscripcionTorneoTest
HU-SOC-04	Club, Torneo, Posicion	HuSoc04ConsultaTorneosTest
HU-SOC-05	Bono, Venta	HuSoc05BonoTest
HU-EMP-01	Club, Turno, Torneo	HuEmp01InscripcionEmpleadoTest

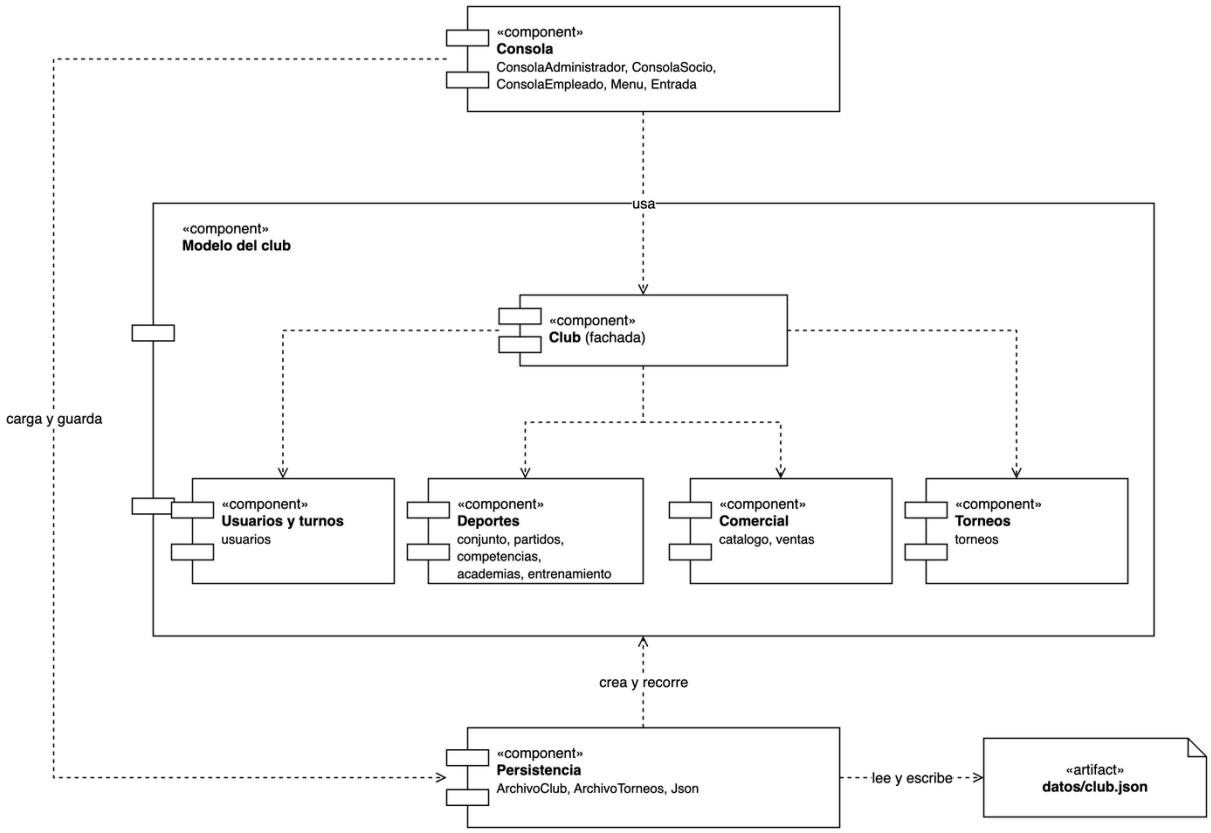
2. Visión general

2.1 Diagrama de contexto



Visto desde afuera, el sistema tiene dos tipos de actores/recursos con los que interactúa: las personas y la carpeta de datos. Las personas son el administrador, los socios, los estudiantes, los externos, los entrenadores y los fisioterapeutas, cada uno entra por la consola de su rol (administrador, socio o empleado) después de autenticarse. La carpeta de datos es el único recurso externo: el sistema lee el archivo club.json al iniciar y lo escribe al salir para no perder información.

2.2 Diagrama de componentes



2.3 Razonamiento de este diseño

La interfaz está aparte de la lógica, porque responde al RNF02: en el Proyecto 2 la consola pasó de programas de demostración a menús interactivos por rol, y esto se hizo sin cambiar las reglas de negocio. Si mezcláramos la lógica con la forma en la que se le presenta al usuario, existiría un acoplamiento muy tedioso de solucionar a medida que avanza el proyecto.

La persistencia también esta aparte de la lógica y responde al RNF03. Así como las reglas de negocio pueden cambiar, también puede cambiar la forma de persistencia, quizás si se escala se migre a una base relacional u otras opciones. Si el formato cambia o evoluciona, solo cambia este componente.

Deportivo aparte de lo comercial ya que el problema a solucionar ya describe al club como “dos frentes de operación”. Tienen reglas que no se parecen (cupos, niveles y suspensiones de un lado, impuestos, descuentos e inventario del otro) y cambian por razones distintas. Además, permite repartir el trabajo del equipo sin que dos personas editen las mismas clases.

Usuarios es un componente individual porque deben de existir distintos usuarios con comportamientos distintos. El sistema necesita saber quién es la persona: un socio juega y también compra. Si Usuarios viviera dentro de uno de los otros componentes, el otro tendría que depender de un frente que no le corresponde.

Torneos es un componente propio porque sus reglas (cupos preferenciales, tarifas, premios y bonos) no son ni del frente deportivo ni del comercial, aunque usa a ambos: inscribe equipos y jugadores del lado deportivo y entrega bonos que se usan en las ventas. Así, un cambio en las reglas de los torneos no obliga a tocar los demás componentes.

3. Roles y candidatos

3.1 Clases de la aplicación

Clase	Estereotipo	Descripción
Club	«coordinator»	Guarda las listas y recibe las solicitudes delega las reglas en quien tiene la info
ArchivoClub, ArchivoTorneos	«interfacer»	Guardan y cargan toda la información del club, incluidos los torneos, en datos/club.json
Json	«service provider»	Convierte entre texto JSON y mapas o listas de Java, la usan todas las clases de persistencia
ConsolaBase	«controller»	Carga los datos, autentica al usuario, muestra el menú de su rol y guarda al salir
ConsolaAdministrador, ConsolaSocio, ConsolaEmpleado	«controller»	Programa (main) de cada tipo de usuario, arman el menú con las opciones de su rol
Menu, Accion	«structurer»	Menú numerado reutilizable, cada opción es una Accion que puede fallar por una regla de negocio

Entrada	«service provider»	Lee y valida todas las entradas: números, fechas, horas, s/n y listas
MenuAdminPersonal, MenuAdminDeportes, MenuAdminTienda, MenuAdminTorneos, MenuEntrenador, MenuFisioterapeuta, MenuCompras, MenuTorneosJugador, RegistroPartido	«controller»	Opciones de cada área: piden los datos con Entrada y llaman a Club
VistaTorneos	«service provider»	Imprime listas de torneos, tablas de posiciones e historial

3.2 Clases de dominio

Paquete	Clase	Estereotipo	Descripción
usuarios	Usuario	«information holder»	Login, password, nombre y puntos
usuarios	Socio	«information holder»	Sabe su edad y si está activo y habilitado
usuarios	Empleado, Entrenador, Fisioterapeuta	«information holder»	Empleados con su código de descuento
usuarios	Administrador	«information holder»	Administra

usuarios	RestriccionMedica	«information holder»	Periodo en que un socio no puede jugar y la persona que lo registro
usuarios	Turno, SolicitudCambioTurno	«information holder»	Franja de trabajo y pedido de cambio
conjunto	Equipo	«structurer»	Organiza jugadores y suspensiones, y logica de inscripcion etc
conjunto	MiembroEquipo	«information holder»	Un socio en un equipo, con posición y camiseta
conjunto	PartidoInterno	«structurer»	Partido entre equipos del club, valida el mínimo de jugadores y crea suspensiones
partidos	Partido	«structurer»	Pasos comunes: participantes, estadísticas y cierre
partidos	EstadisticaIndividual, EstadisticaFutbol	«information holder»	Lo que hizo un jugadorl
partidos	EstadisticaBaloncesto, EstadisticaVoleibol, EstadisticaRaqueta, EstadisticaTenisMesa	«information holder»	Estadísticas de cada deporte
partidos	Suspension	«information holder»	Sanción para el siguiente partido del equipo

competencias	Competencia, Liga, CompetenciaPorFases	«structurer»	Torneo externo, clasificacion depende del formato
competencias	Fase, NominaCompetencia, PartidoCompetencia	«structurer»	Etapas, jugadores inscritos y partidos oficiales
academias	Academia, PartidoPractica	«structurer»	Inscripciones e instalaciones,
academias	InscripcionAcademia, RegistroRanking, Instalacion, Reserva	«information holder»	Nivel y ranking, su historial, canchas y su uso
entrenamiento	SesionEntrenamiento	«structurer»	Sesión con convocados
entrenamiento	Asistencia, EvaluacionDesempeno	«information holder»	Asistencia de un convocado y su nota
catalogo	Vendible (interfaz)	«information holder»	Todo lo que se puede vender: tiene nombre y precio
catalogo	Articulo	«information holder»	Artículo de la tienda con categoría y precio
catalogo	ProductoCafeteria, Bebida, Snack	«information holder»	Productos de la cafetería con sus alérgenos, la bebida sabe si es caliente
catalogo	Inventario	«information holder»	Unidades de un artículo en una ubicación, valida que alcance al retirar
catalogo	Ubicacion	«information holder»	Lugar donde se guarda inventario (bodega, tienda)

catalogo	Sugerencia	«information holder»	Propuesta de producto nuevo (artículo o snack, con precio y alérgenos) hecha por un empleado, con su estado
ventas	TiendaCafeteria	«structurer»	Reúne catálogo, inventario, ventas, sugerencias y mensualidades, traslada unidades
ventas	Venta	«service provider»	Compra genérica: calcula subtotal, total y puntos
ventas	VentaTienda	«service provider»	Aplica IVA, código de descuento y descuenta el inventario
ventas	VentaCafeteria	«service provider»	Aplica impoconsumo y propina, informa alérgenos y valida RN-20
ventas	LineaVenta	«information holder»	Un producto de la venta con su cantidad y el precio de ese día
ventas	Mensualidad	«information holder»	Cuota mensual de un socio y su fecha de pago
usuarios	Estudiante, Externo	«information holder»	Participantes de torneos que no son socios, el administrador los registra
torneos	Torneo	«structurer»	Cupos preferenciales y regulares, inscripciones, encuentros, tabla de posiciones y cierre
torneos	TorneoRecreativo, TorneoCompetitivo	«service provider»	Deciden la tarifa y el premio: bono del 15% o 70% de lo recaudado
torneos	Especialidad, EspecialidadConjunto, EspecialidadIndividual	«information holder»	Deporte y categoría, o disciplina y nivel, validan quién se puede inscribir

torneos	Participante (interfaz), JugadorTorneo, ParejaTorneo, EquipoTorneo	«information holder»	Quien ocupa un cupo: un jugador, una pareja o un equipo
torneos	InscripcionTorneo, EncuentroTorneo, Posicion	«information holder»	Cupo ocupado y monto pagado, resultado de un encuentro y fila de la tabla de posiciones
torneos	RankingTorneos	«service provider»	Decide si un participante está entre los mejores de su categoría
ventas	Bono	«information holder»	Descuento ganado en un torneo recreativo, se usa una sola vez
ventas	ResumenVentas	«information holder»	Totales de un rubro en un periodo para el reporte de ingresos

Enum: DeporteConjunto, CategoriaEdad, DiaSemana, EstadoSolicitud, TipoFase, Disciplina, Nivel, Modalidad, TipoInstalacion, EstadoAsistencia, Alergeno, Estadosugerencia,Rubro, TipoSugerencia, EstadoTorneo, FormatoTorneo.

3.3 Descartados

Candidato	**Razon de descarte**
Tarjeta, TipoTarjeta	Solo se cuentan dos int en EstadisticaFutbol
EstadoPartido	Dos valores boolean jugado en Partido

Ranking	Es por jugador y disciplina, puede estar en InscripcionAcademia
Deporte, Categoria, Nivel como clases	Valores fijos que solo cambian en algunos datos entonces mejor se usa un enum
Posición dentro de un equipo, CategoriaArticulo	Solo un string
PuntosFidelidad, CodigoDescuento	Un número y un texto, entonces puede ir en atributos
Servicio de autenticación aparte	Inicialmente es manejable desde Club
Producto (clase padre abstracta)	Artículos y productos de cafetería no comparten atributos, solo que se venden, se usa la interfaz Vendible
Temperatura (enum)	Solo hay dos valores, basta un boolean caliente en Bebida
Eliminación directa en los torneos	Todos contra todos con tabla de posiciones reutiliza la idea de Liga y siempre deja un orden final

4. Responsabilidades

#	**Responsabilidad**	**Tipo**	**Asignacion**
1	Saber los usuarios, equipos y partidos	Saber	Club
2	Rechazar un login repetido y autenticar	Decidir	Club

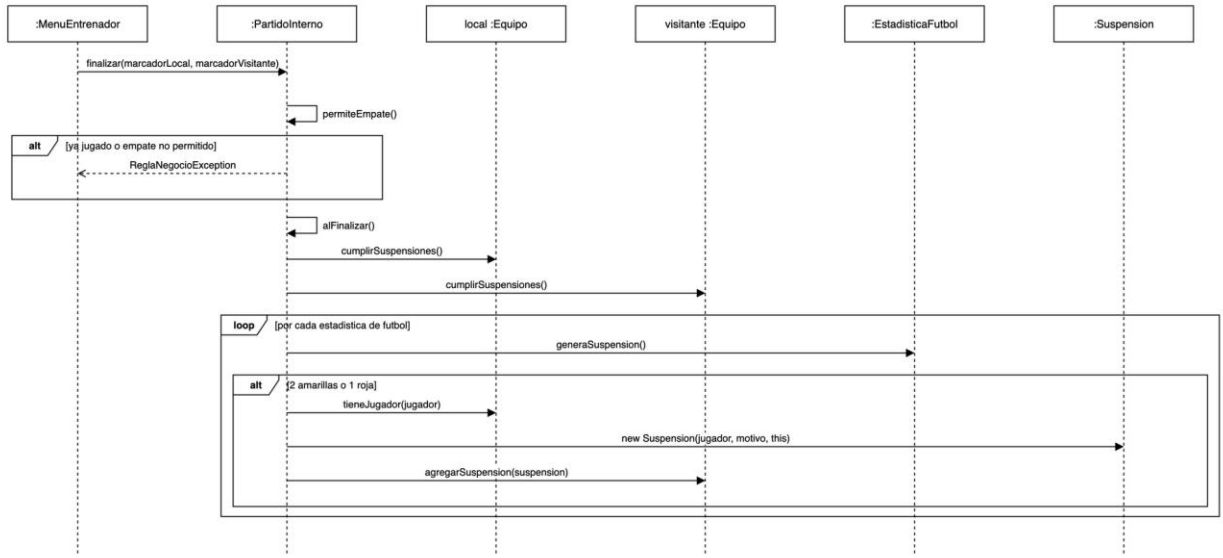
3	Verificar un password	Decidir	Usuario
4	Calcular la edad de un socio a una fecha	Hacer	Socio
5	Decidir si un socio está habilitado	Decidir	Socio
6	Registrar y dar de alta una restricción médica	Hacer	Club la crea, RestriccionMedica la da de alta
7	Rechazar dos equipos del mismo deporte para un socio	Decidir	Club
8	Validar cupo, camiseta, edad y socio activo al inscribir	Decidir	Equipo
9	Decidir si un socio puede alinearse	Decidir	Equipo
10	Contar los habilitados de un equipo	Hacer	Equipo
11	Validar los equipos y el mínimo de un partido interno	Decidir	PartidoInterno
12	Aceptar participantes y estadísticas solo antes del cierre	Decidir	Partido
13	Decidir quién puede participar	Decidir	subclase de Partido
14	Rechazar un empate donde no se permite	Decidir	Partido, preguntándole a la subclase

15	Decidir si una estadística genera suspensión	Decidir	EstadisticaFutbol
16	Crear y cumplir suspensiones al terminar un partido	Hacer	PartidoInterno
17	Guardar las suspensiones pendientes	Saber	Equipo
18	Leer y validar las entradas de la consola	Hacer	Entrada
19	Decidir si un socio ya esta inscrito en una academia	Decidir	Academia
20	Validar capacidad, cupo simultaneo y choque de horario al reservar	Decidir	Academia
21	Decidir si dos jugadores son compatibles en nivel para un partido de practica, salvo cuando alguno de los dos es entrenador, caso en el cual la restricción de nivel no aplica	Decidir	PartidoPractica
22	Actualizar el puntaje de ranking tras un resultado	Hacer	Inscripcion Académica
23	Validar requisitos de un equipo para inscribirse en una competencia	Decidir	Competencia
24	Decidir quien puede participar en un partido oficial	Decidir	PartidoCompetencia
25	Decidir si un partido oficial permite empate	Decidir	PartidoCompetencia
26	Calcular el puntaje/clasificación tras un resultado oficial	Decidir	Liga/CompetenciaPorFases
27	Saber el catálogo, el inventario, las ventas, las sugerencias y las mensualidades	Saber	TiendaCafeteria

28	Rechazar artículos, productos o ubicaciones con nombre repetido	Decidir	TiendaCafeteria
29	Sumar y retirar unidades, validando que alcance el stock	Decidir	Inventario
30	Trasladar todo el inventario de un artículo a otra ubicación donde no esté	Hacer	TiendaCafeteria
31	Decidir que producto acepta cada tipo de venta	Decidir	VentaTIenda,VentaCafeteria
32	Calcular subtotal y total de una venta	Hacer	Venta
33	Calcular el descuento por código (15% dueño, 8% compartido, no acumulable)	Decidir	VentaTienda
34	Calcular el impuesto (IVA 19% o impoconsumo 8%)	Hacer	VentaTienda, VentaCafeteria
35	Calcular la propina y permitir cambiarla	Hacer	VentaCafeteria
36	Reunir los alérgenos de los productos de la venta	Hacer	VentaCafeteria, preguntándole a ProductoCafeteria
37	Rechazar una bebida caliente en pádel o tenis de mesa techada (RN-20)	Decidir	VentaCafeteria, preguntándole a ProductoCafeteria
38	Calcular los puntos generados y validar los redimidos	Decidir	Venta
39	Descontar del inventario lo vendido	Hacer	VentaTienda
40	Registrar el pago de una mensualidad una sola vez	Decidir	Mensualidad
41	Aprobar o rechazar solo sugerencias pendientes, y crear el producto al aprobar	Decidir	Sugerencia, TiendaCafeteria
42	Guardar y cargar todo el club, incluidos los torneos	Hacer	ArchivoClub, ArchivoTorneos

43	Convertir entre texto JSON y estructuras de Java	Hacer	Json
44	Autenticar al usuario y mostrar el menú de su rol	Hacer	ConsolaBase y sus subclases
45	Construir menús numerados reutilizables	Hacer	Menu
46	Validar una alineación completa antes de registrarla	Decidir	Partido
47	Listar los jugadores no disponibles con su motivo	Hacer	Equipo
48	Calcular los cupos preferenciales (20% redondeado hacia arriba) y liberarlos	Decidir	Torneo
49	Decidir si un participante está entre los mejores de su categoría	Decidir	RankingTorneos
50	Rechazar empleados con turno el día del torneo o en torneos de conjunto	Decidir	Club
51	Validar que el participante corresponde a la especialidad del torneo	Decidir	Especialidad
52	Calcular la tarifa de inscripción, gratis para empleados	Hacer	TorneoCompetitivo
53	Generar los encuentros y la tabla de posiciones	Hacer	Torneo, Posicion
54	Entregar el premio: bono del 15% o 70% de lo recaudado	Hacer	TorneoRecreativo, TorneoCompetitivo
55	Aplicar un bono sin acumularlo con otros descuentos	Decidir	Venta
56	Usar un bono una sola vez	Decidir	Bono

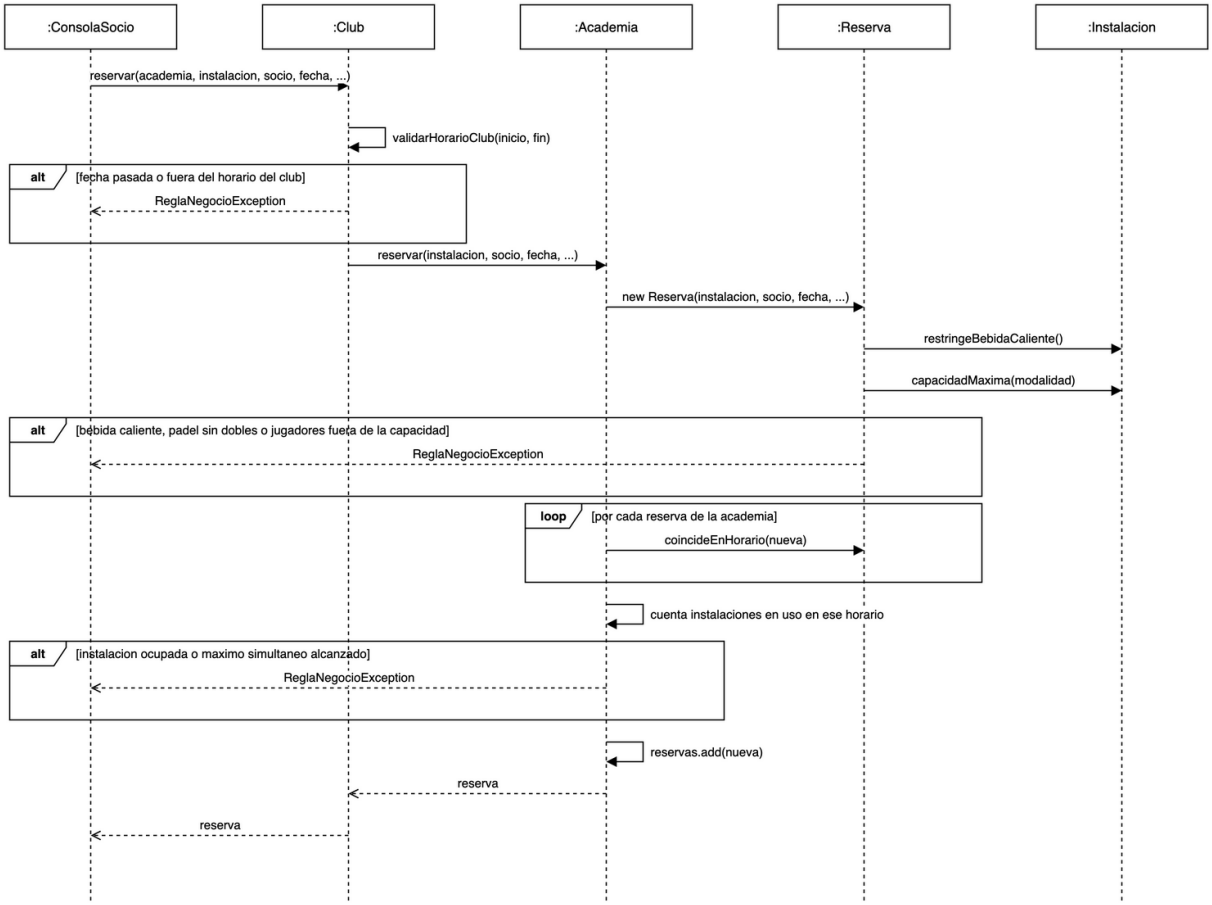
5 Colaboraciones y Control



Escenario: Cerrar un Partido interno

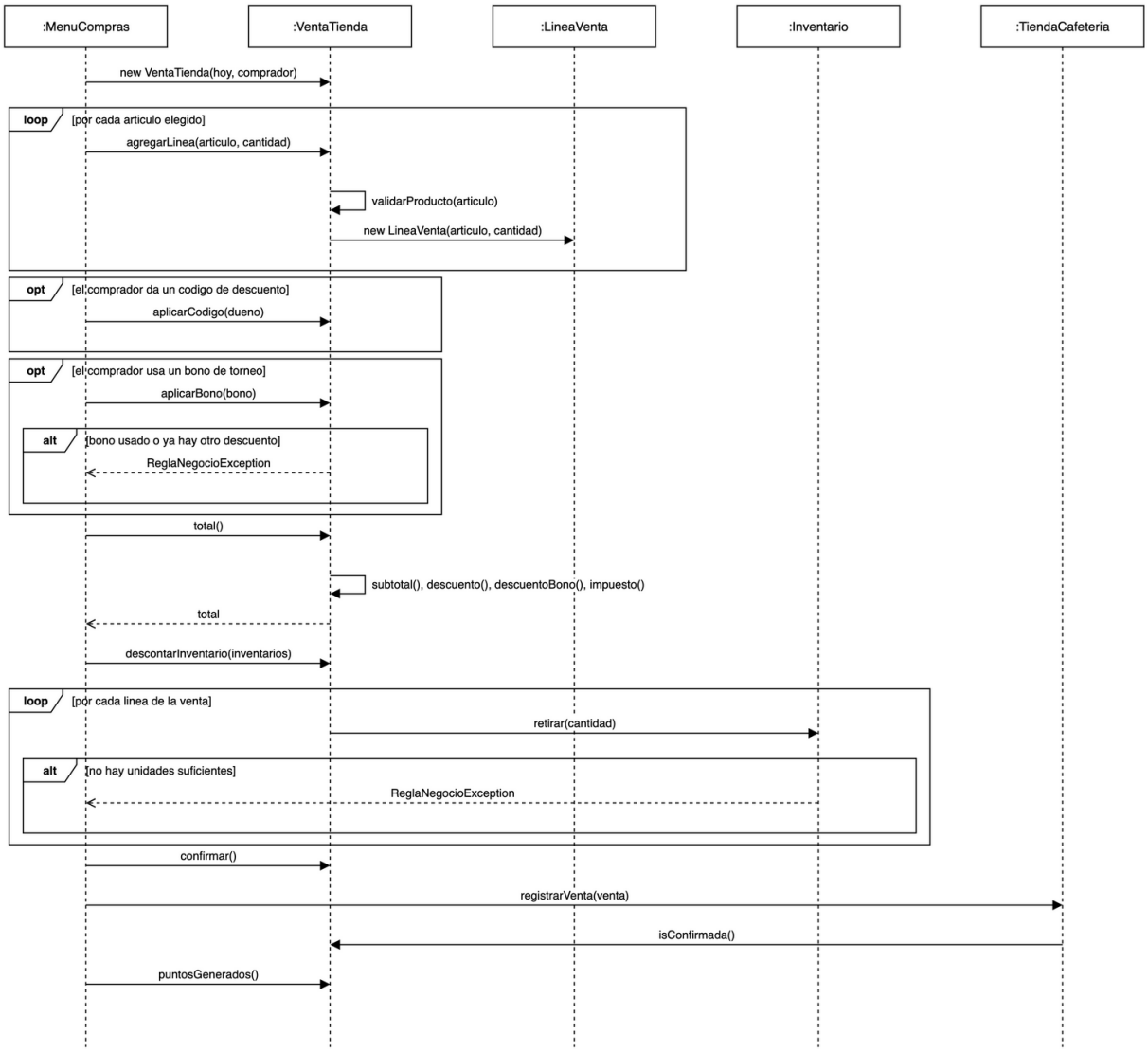
Este escenario es importante porque junta varias cosas a la vez: usa el método alFinalizar(), que es un espacio vacío que Partido deja a propósito para que cada subclase lo llene a su manera, revisa las estadísticas del partido para ver si algún jugador se ganó una suspensión, y si es así, crea objetos Suspensión nuevos que van a afectar a los próximos partidos de ese jugador. Quien recibe la orden de cerrar el partido es PartidoInterno, y es el mismo quien decide, a través de su alFinalizar(), que jugadores quedan suspendidos, pero no se pone a manejar suspensiones

directamente, sino que le pasa la responsabilidad a casa Equipo, este le dice que cumpla las suspensiones que ya tenía pendiente y que agregue las nuevas, sin meterse en como el equipo las guarda por dentro. Por eso el estilo de control es delegado: PartidoInerno coordina el cierre del partido, pero reparte el trabajo en vez de hacerlo todo el mismo

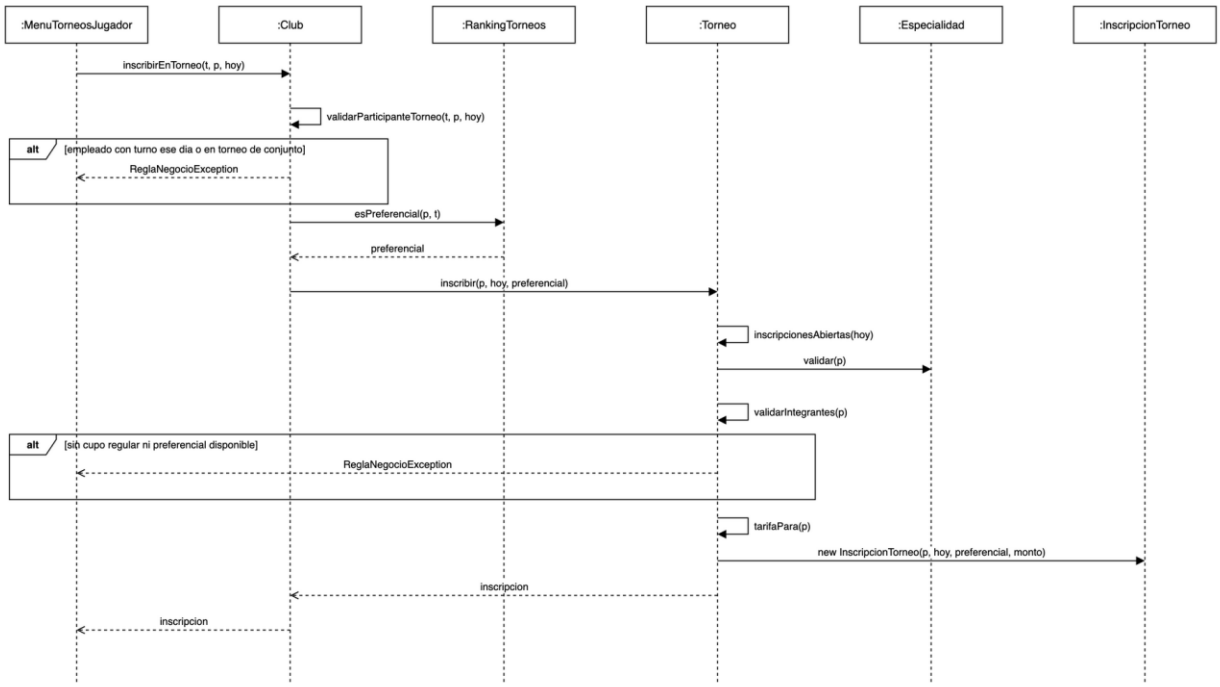


Escenario: Reservar una instalación en una academia

Este diagrama muestra cómo se valida una reserva de instalación en una academia. Cuando un socio pide reservar desde su consola, Club valida primero que la fecha no haya pasado y que esté dentro del horario del club, después la Academia confirma que esa instalación sea suya, y al crear la Reserva se valida de una que el número de jugadores quepan en la capacidad de esa instalación segun la modalidad que el pádel sea en dobles, que el socio no lleve una bebida caliente a pádel o tenis de mesa, y que la hora de fin sea después que la de inicio. Ya con la reserva creada, la Academia revisa dos reglas: que esa instalación no esté ya ocupada en ese mismo horario por otra reserva, y que no se haya llegado al máximo de instalaciones que pueden estar en uso al mismo tiempo. Solo si pasa las reglas se guarda la reserva y se devuelve.



En este ecenario es un punto creitico ya que con una sola operacion se estan aplicando muchas reglas de el negocio : el código de descuento (15% para el dueño, 8% si es compartido), el IVA del 19%, la verificación del stock , los puntos de fidelidad y el bono de torneo, que no se acumula con el código ni con los puntos. Esta solicitud se recibe por medio de VentaTienda, ve esta valida cada prodicto pues con ValidarProducto() y le pide al inventario que retire las unidades ya vendidas , si alguno no tiene suficientes , el inventario lanza un ReglaNegocioExeption y la venta no se termina completando . Para calcular el total , venta define el algoritmo general (subtotal – descuento + impuestos) y nos deja que VentaTienda decida el descuento y el valor de el impuesto . El estido de control es delegado: VentaTienda coordina la venta, pero cada objeto decide sobre lo que conoce (el Inventario sobre su stock, la LineaVenta sobre su subtotal y TiendaCafeteria sobre qué ventas registra).



Escenario: Inscribirse en un torneo

Este escenario reúne casi todas las reglas de los torneos en una sola operación. El jugador pide inscribirse desde su consola y Club valida primero que un empleado no tenga turno el día del torneo y que no intente entrar a un torneo de conjunto. Luego le pregunta a RankingTorneos si el participante está entre los mejores de su categoría, y con esa respuesta le pide al Torneo que lo inscriba. El Torneo revisa que las inscripciones sigan abiertas, le pide a su Especialidad que valide al participante, revisa que nadie quede inscrito dos veces y decide el cupo (preferencial si es de los mejores y hay cupo, regular si queda cupo regular, o rechazo si solo quedan cupos preferenciales antes de la fecha de liberación). Por último calcula la tarifa con tarifaPara(), que cada subclase define, y crea la InscripcionTorneo.

6. Diseño de alto nivel

Figura 6.1 Alto nivel: paquetes y sus dependencias

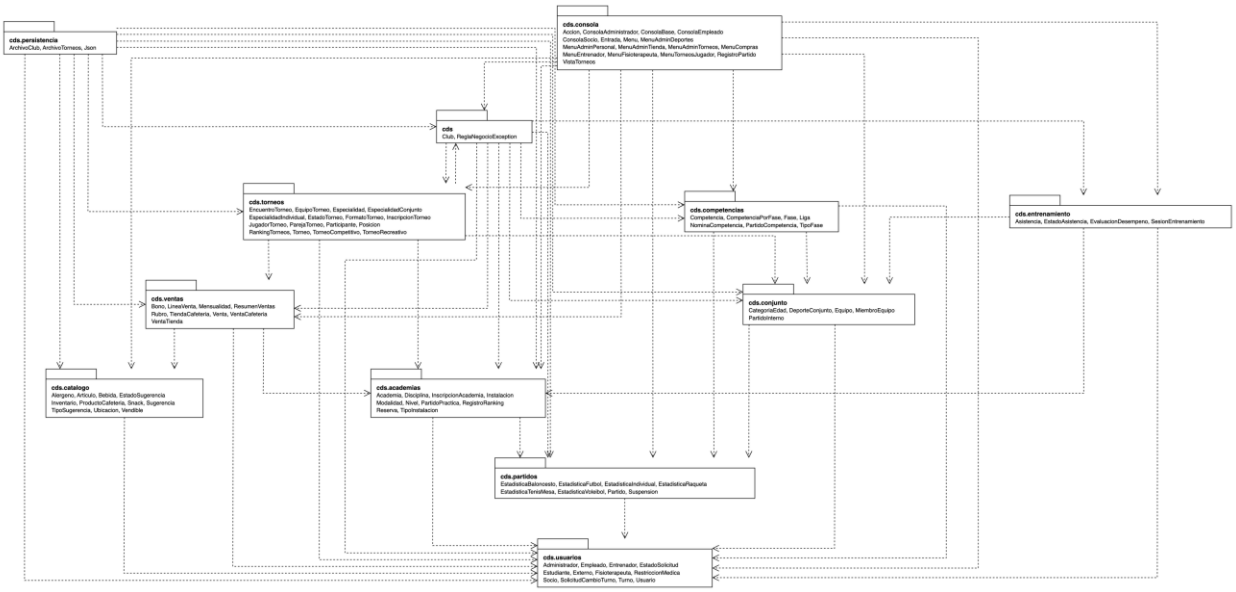


Figura 6.2 Modelo de dominio: Usuarios y turnos

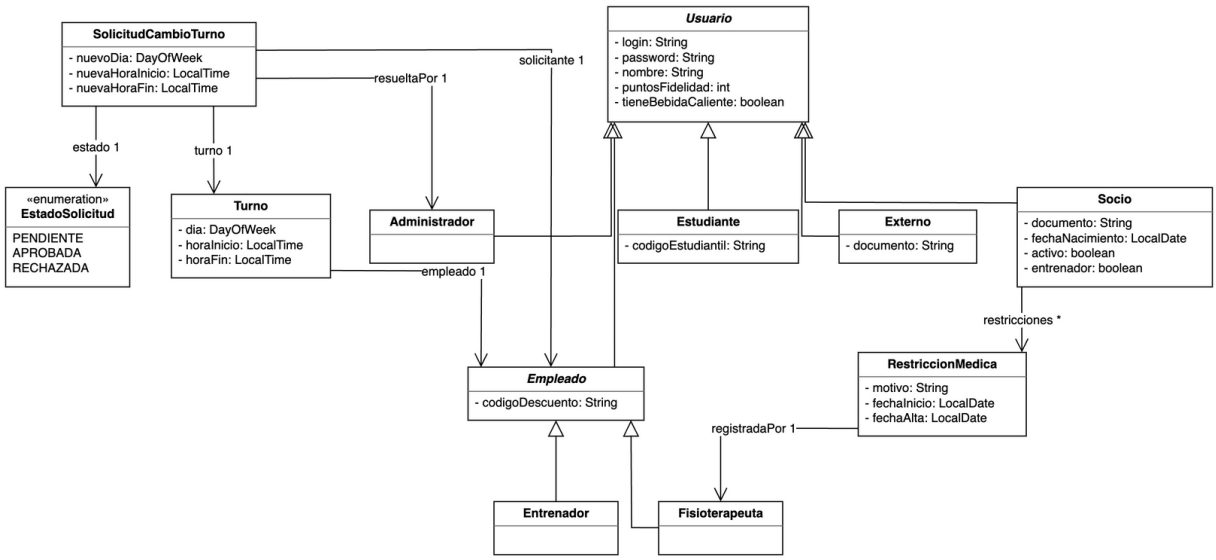


Figura 6.3 Modelo de dominio: Deportes de conjunto

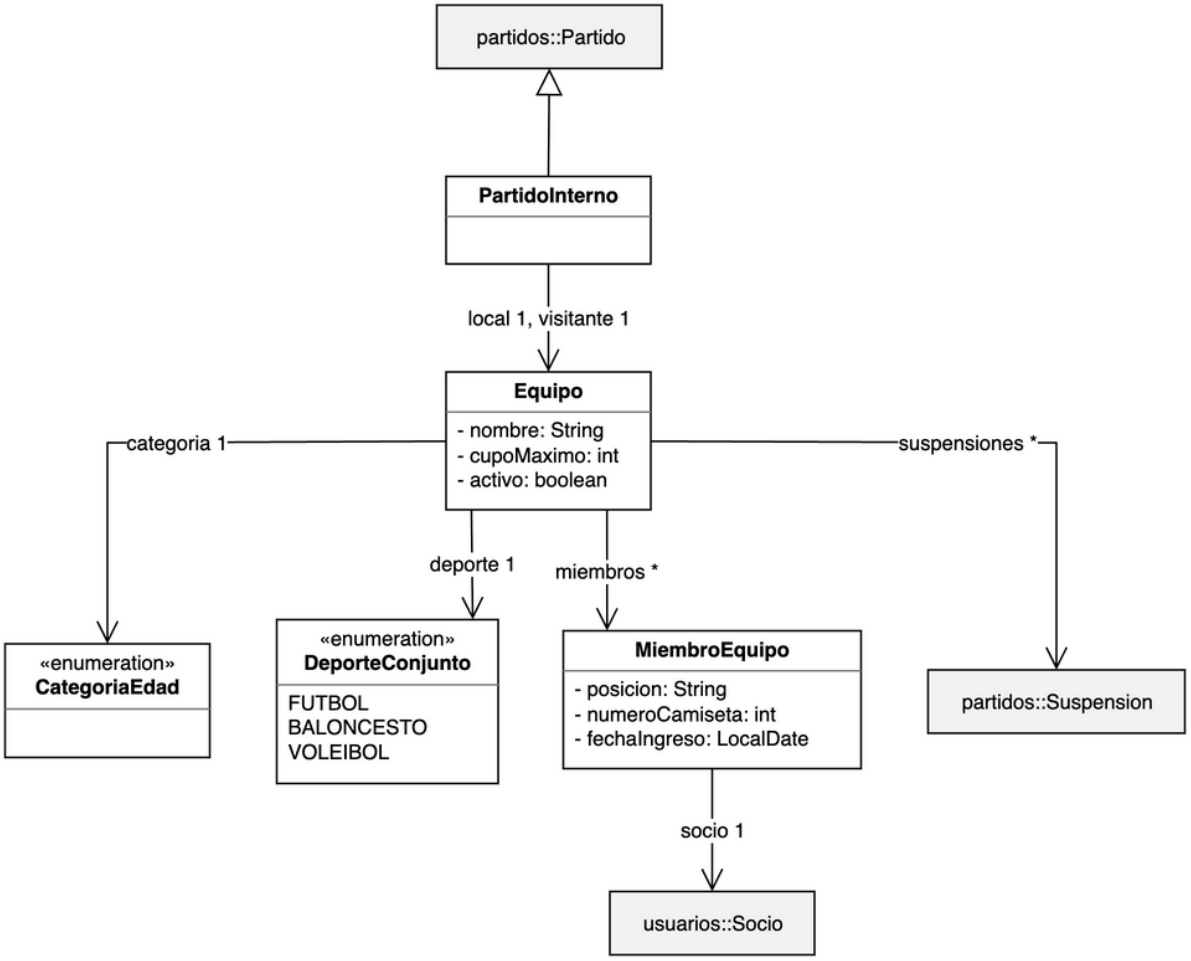


Figura 6.4 Modelo de dominio: Partidos y estadísticas

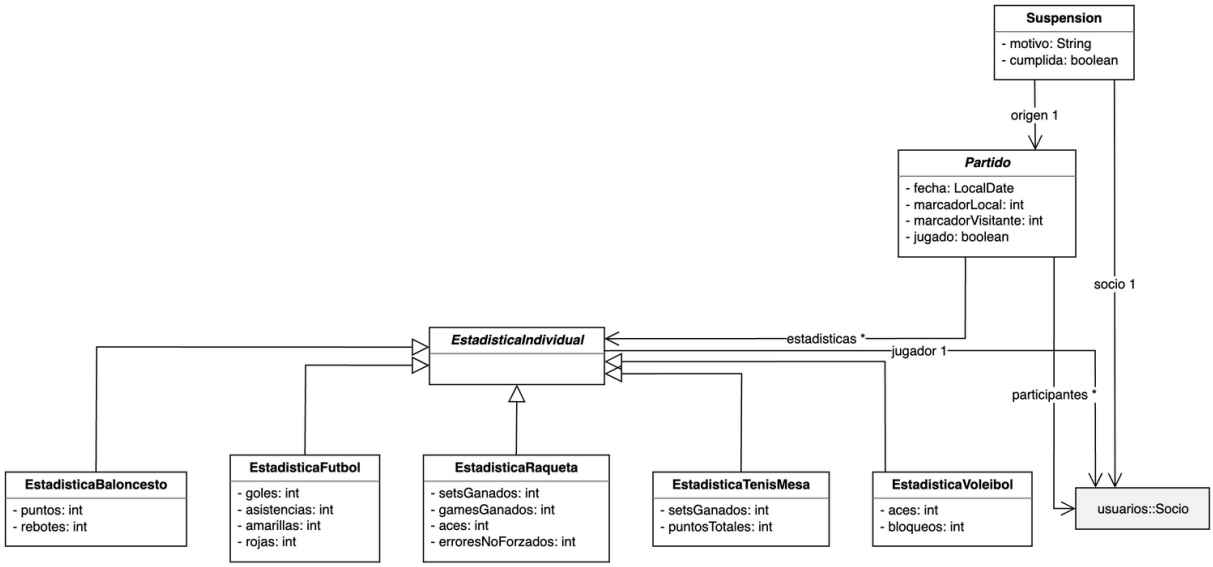


Figura 6.5 Modelo de dominio: Competencias

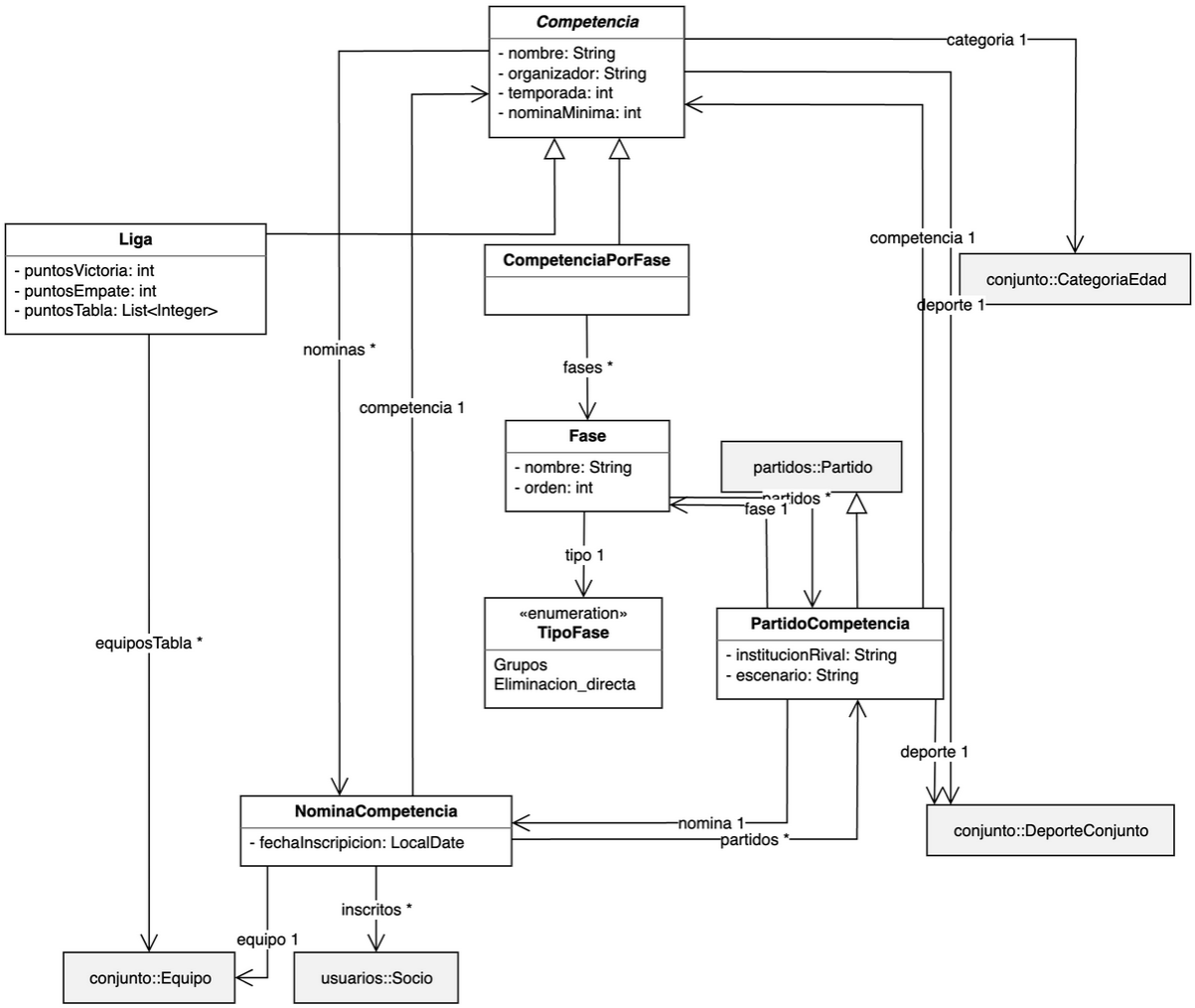


Figura 6.6 Modelo de dominio: Academias

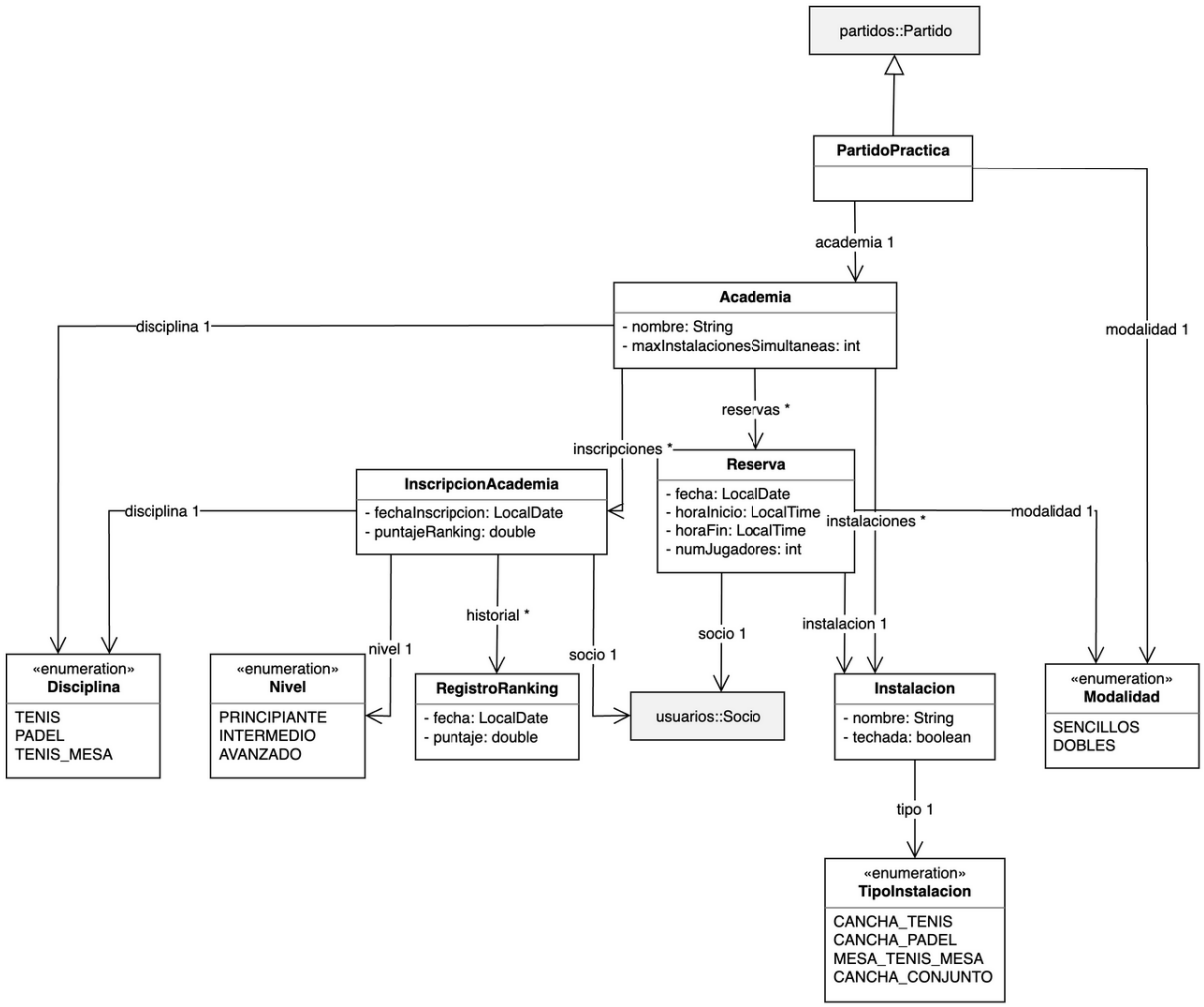


Figura 6.7 Modelo de dominio: Entrenamiento

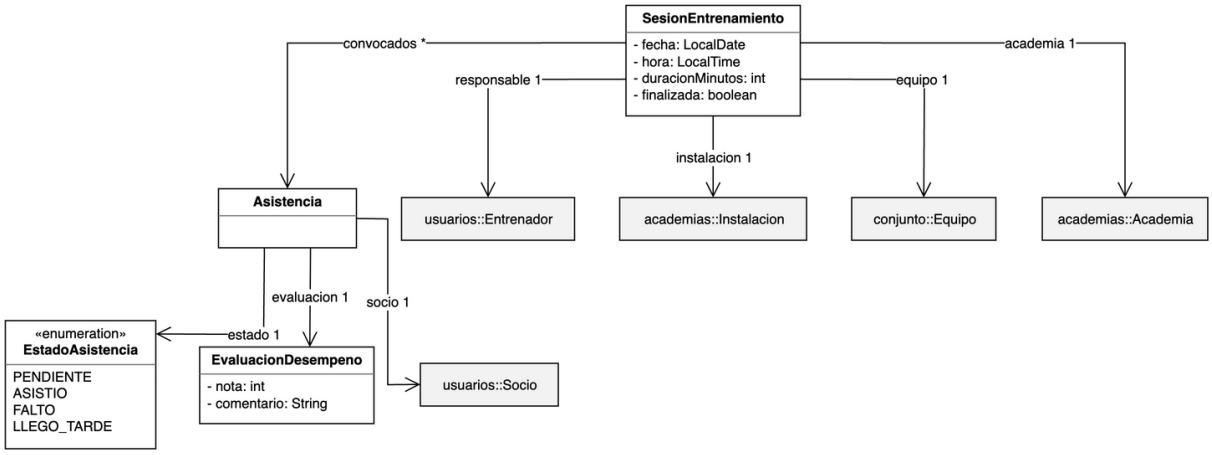


Figura 6.8 Modelo de dominio: Catálogo e inventario

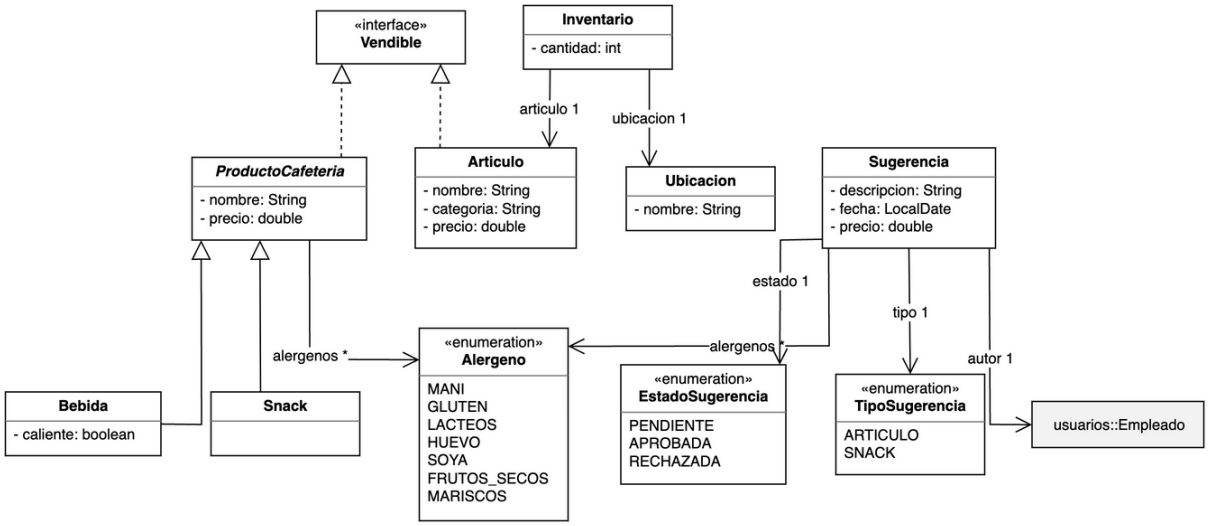


Figura 6.9 Modelo de dominio: Ventas y pagos

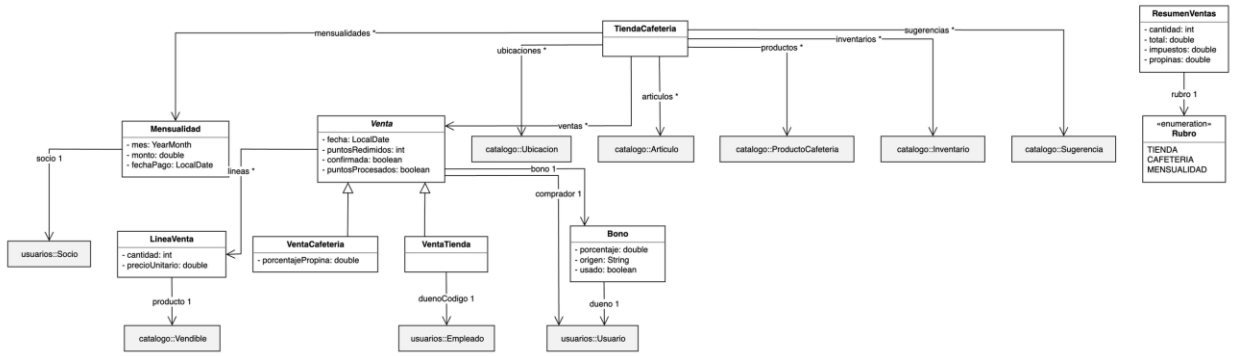
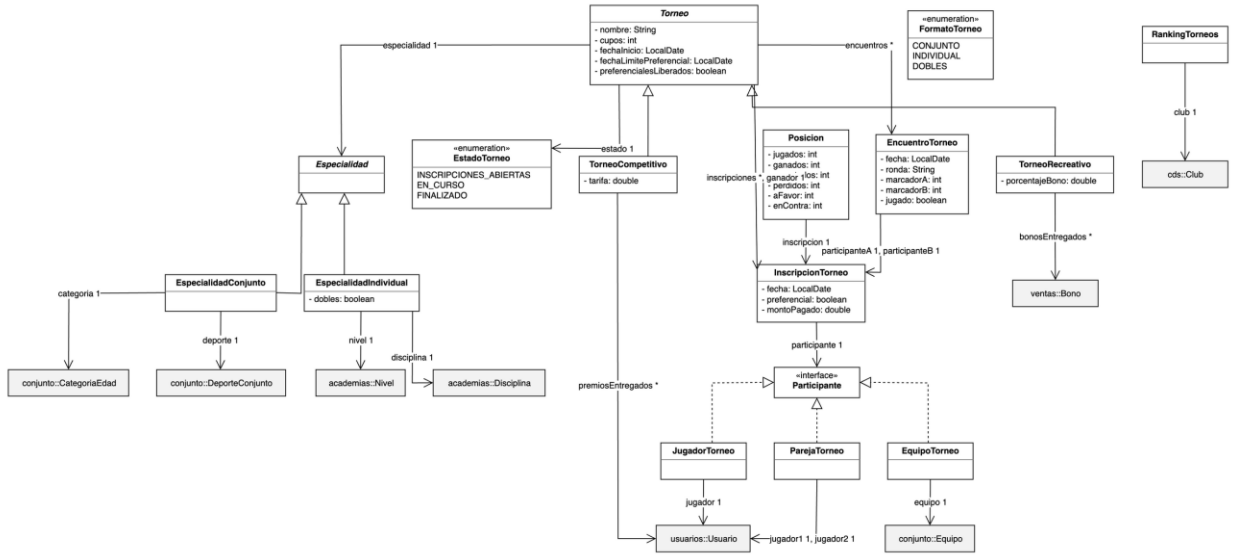


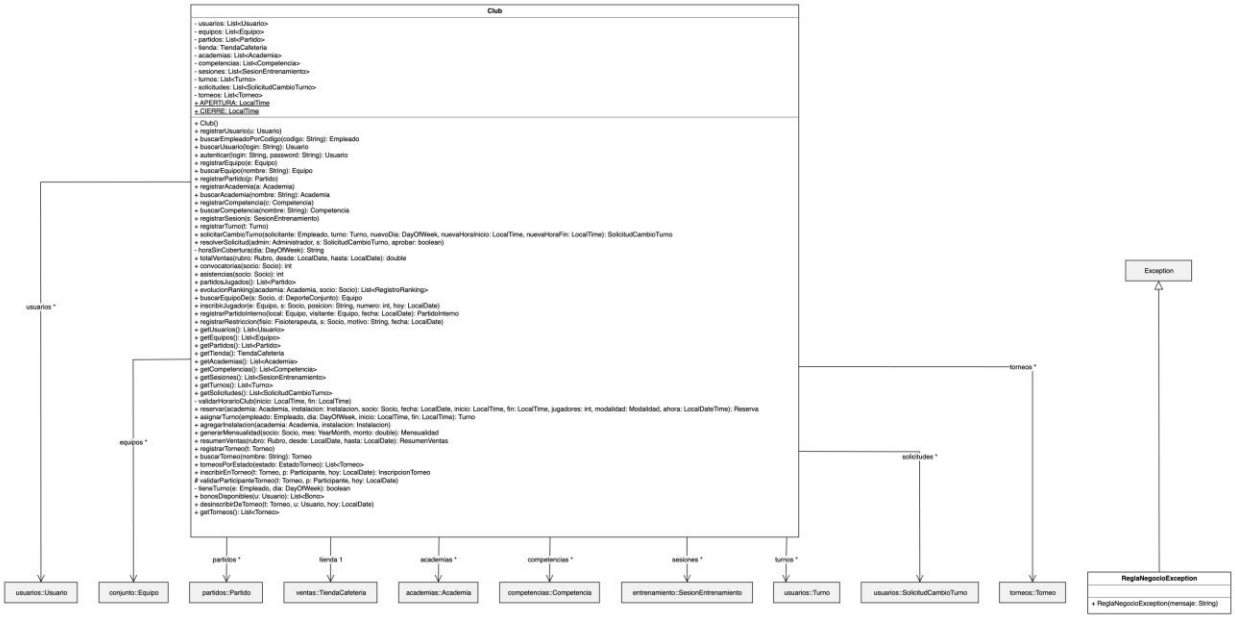
Figura 6.10 Modelo de dominio: Torneos



El diagrama de alto nivel muestra los paquetes del sistema y las dependencias entre ellos: la consola y la persistencia dependen del modelo, y el modelo no depende de ninguna de las dos (RNF-02 y RNF-03). Club, en el paquete cds, es la fachada que reúne a los demás. Las figuras siguientes son el modelo de dominio, una por paquete: muestran cada clase con sus atributos simples y las relaciones con su multiplicidad, sin métodos, que quedan para la sección 7. Las clases de otro paquete aparecen en gris con su nombre calificado (por ejemplo usuarios::Socio), y por eso algunas se repiten en varias figuras: así se ve con qué otros paquetes se relaciona cada uno sin dibujar todo en un solo diagrama. El paquete torneos es nuevo en el Proyecto 2.

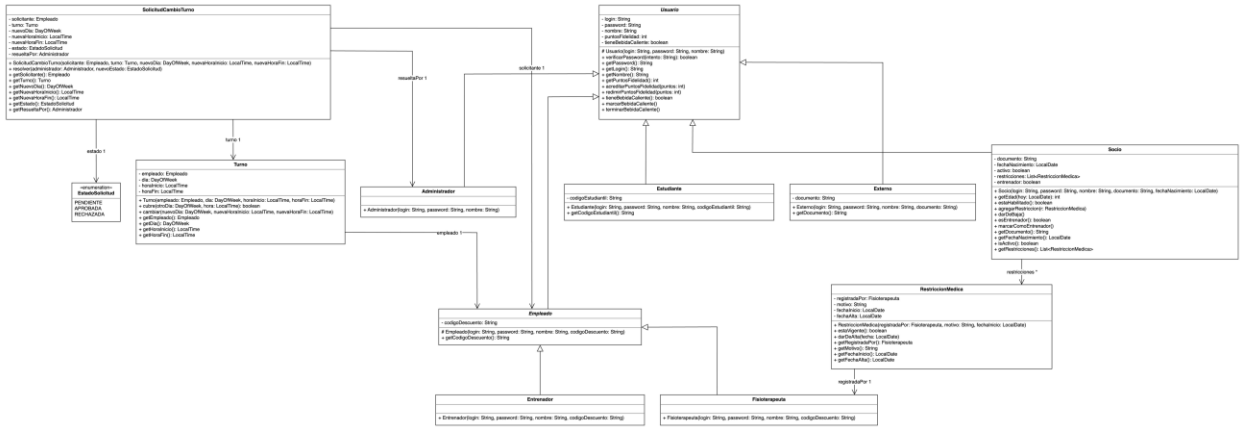
7. Diseño detallado

Figura 7.1 Diagrama de clases detallado: Club



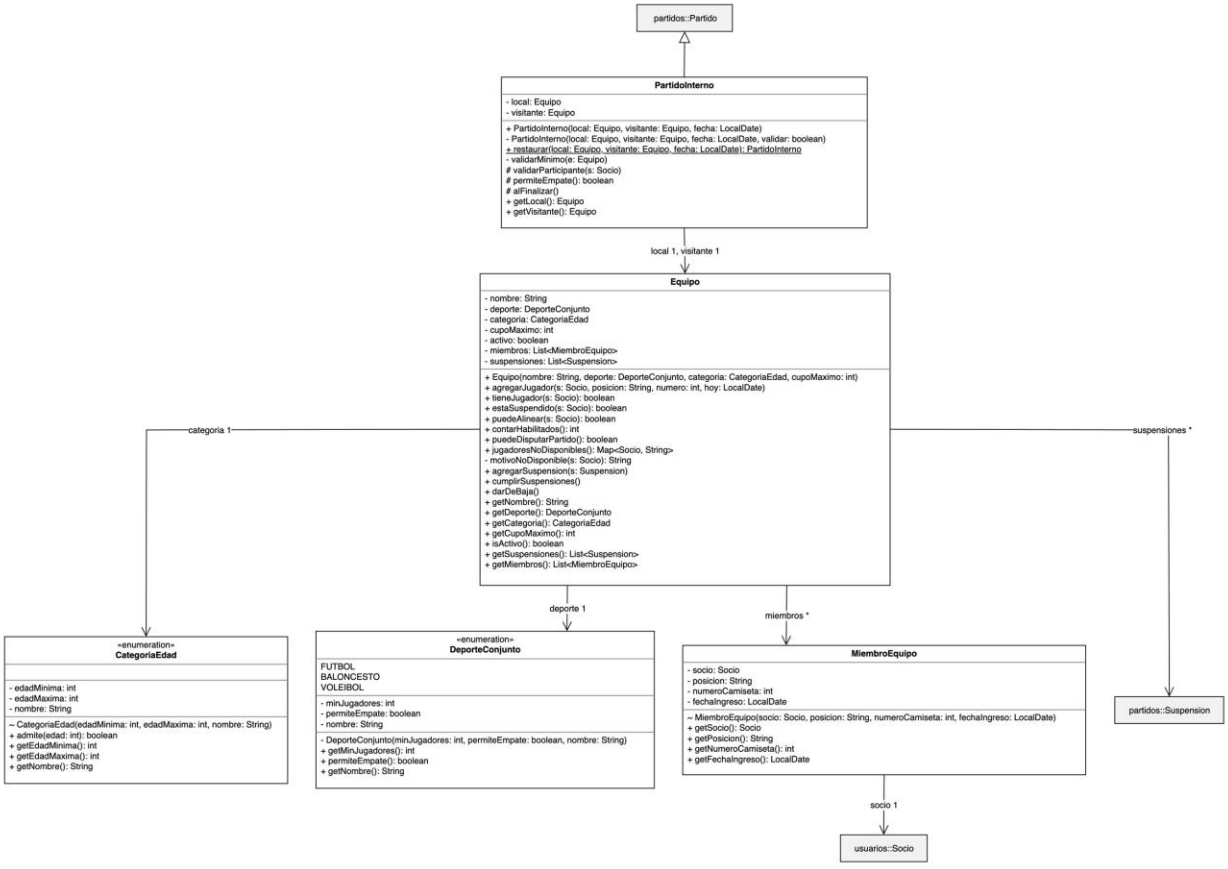
Club es la fachada única del sistema: guarda por composición todo lo del club y una referencia a la tienda, sirviendo de punto de entrada única, aunque internamente cada módulo maneja sus propias reglas ReglaNegocioException es la unica excepción del sistema, usada en todos los módulos para rechazar operaciones invalidas con un mensaje claro

- ☐ Figura 7.2 Diagrama de clases detallado: Usuarios y turnos



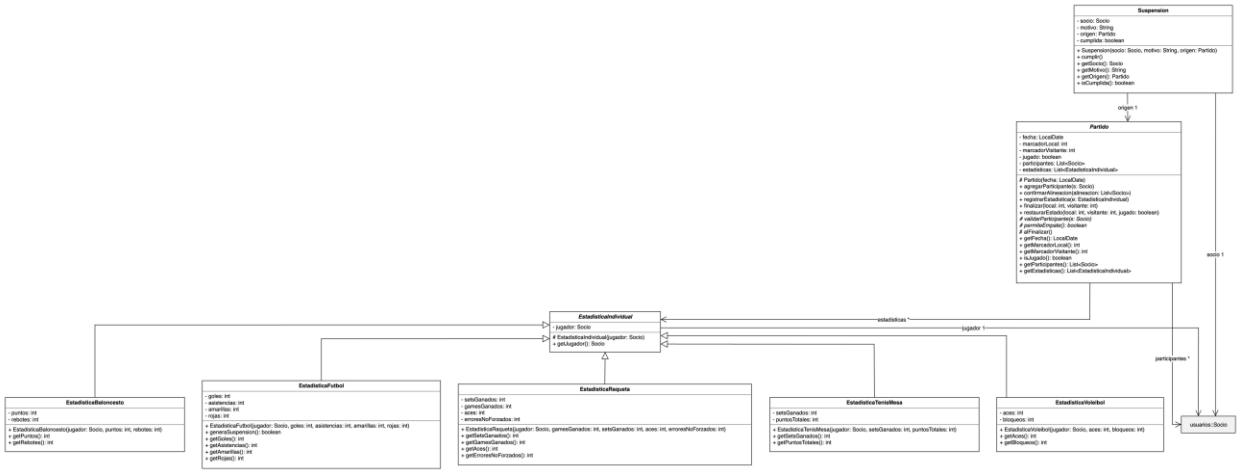
Usuarios es la clase abstracta base (en el Proyecto 2 se agregan Estudiante y Externo, que solo participan en torneos), con tres ramas: Socio, Empleado que se divide Entrenador y Fisioterapeuta, y un Administrador, que hereda directamente de Usuario sin pasar por Empleado. Casa Socio guarda sus propias restricciones medicas por composición, y acá Empleado sus turnos, SolicitudCambioTurno conecta el turno, quien lo pide y quien lo resuelve, con su estado controlado por el enum EstadoSolicitud.

Figura 7.3 Diagrama de clases detallado: Deportes de conjunto



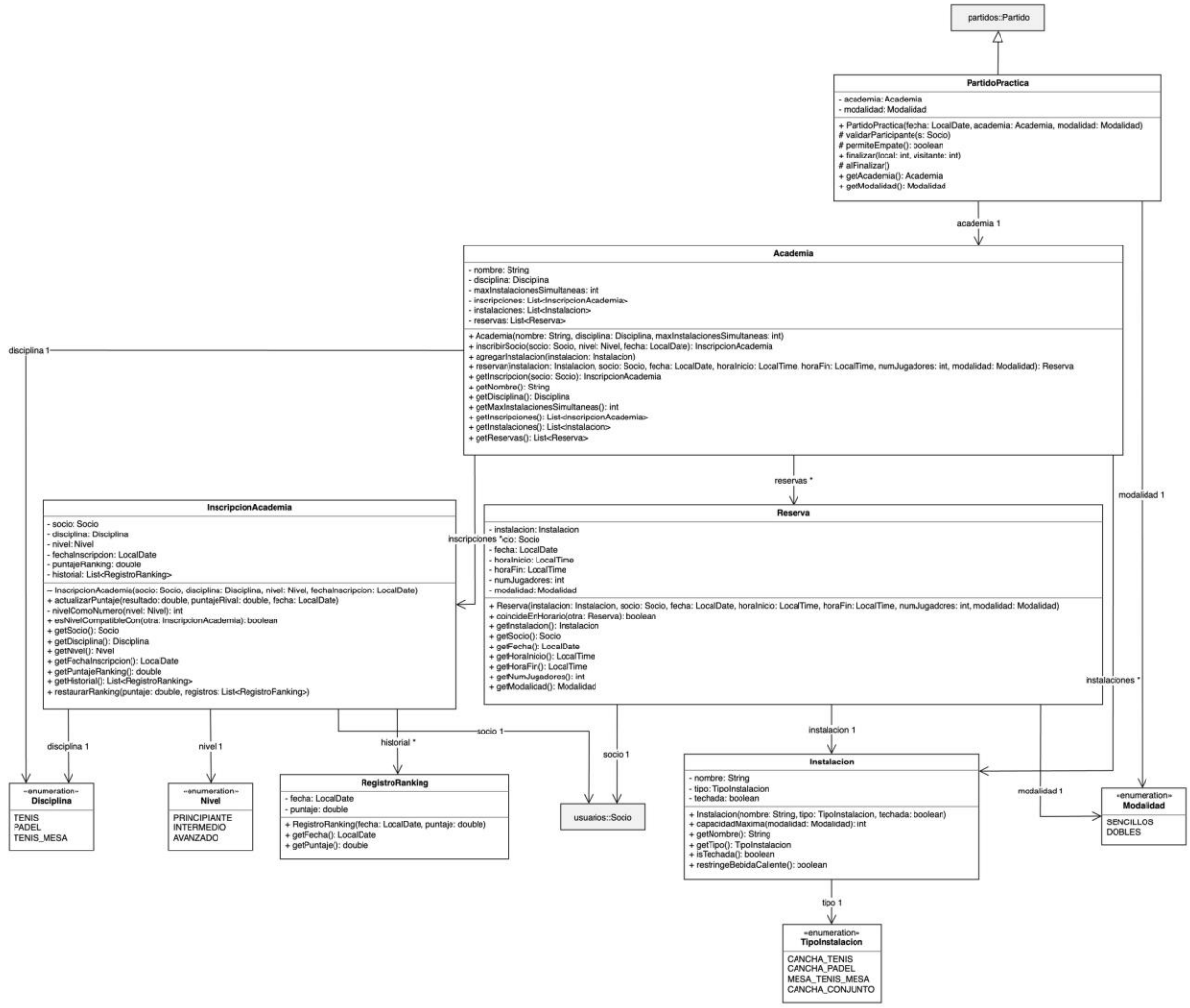
Equipo agrupa a sus jugadores por composición y mantiene sus propias suspensiones pendientes. DeporteConjunto y CategoriaEdad son enums porque son valores fijos con reglas asociadas, no clases con comportamiento propio. PartidoInterno extiende la clase abstracta Partido y solo agrega lo que le es propio: equipo local y visitante

Figura 7.4 Diagrama de clases detallado: Partidos y estadísticas



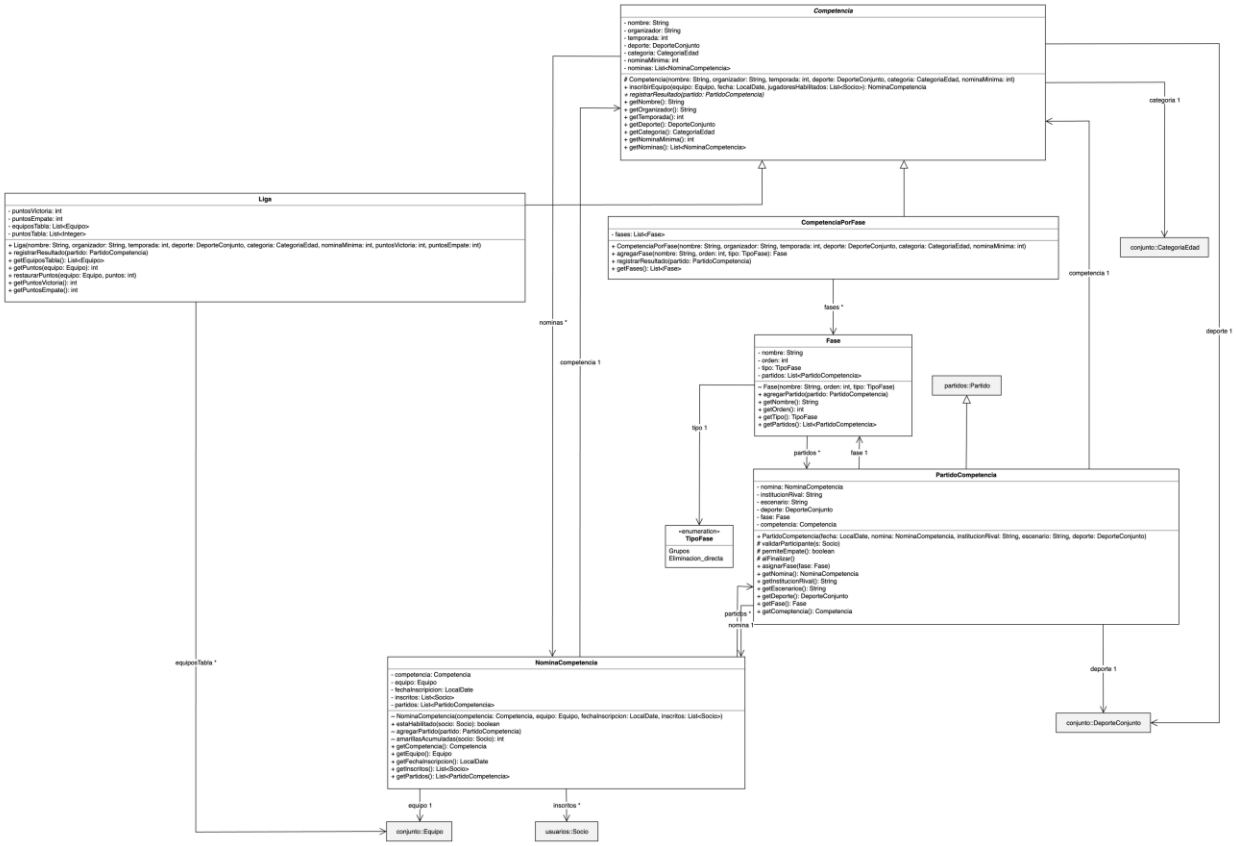
Partido es la clase abstracta que reune lo comun a los tres tipos de partido, PartidoInterno, PartidoCompetencia y PartidoPractica heredan de ella. De la misma forma, EstadisticaIndividual es abstracta y cada deporte tiene su propia subclase con los campos que le interesan, corrigiendo la inconsistencia que tenía el modelo original donde solo algunos deportes heredaban de una clase común.

Figura 7.5 Diagrama de clases detallado: Academias



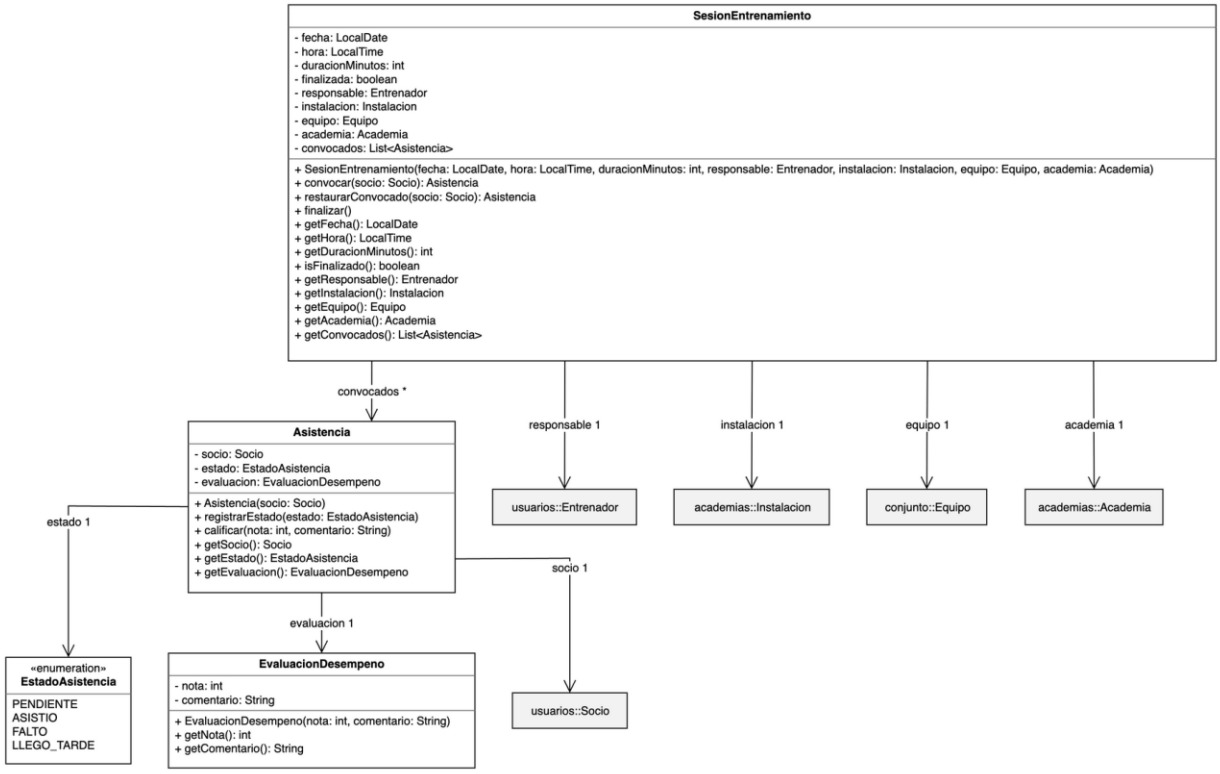
Academia centraliza las inscripciones y las instalaciones disponibles, y valida las reservas que se hacen sobre ellas. Cada InscripciónAcademia guarda el nivel, el puntaje de ranking actual y su historial completo, para poder consultar la evolución del ranking de un socio en el tiempo y no solo su valor actual, PartidoPractica también reutiliza la clase abstracta Partido para su cierre

Figura 7.6 Diagrama de clases detallado: Competencias



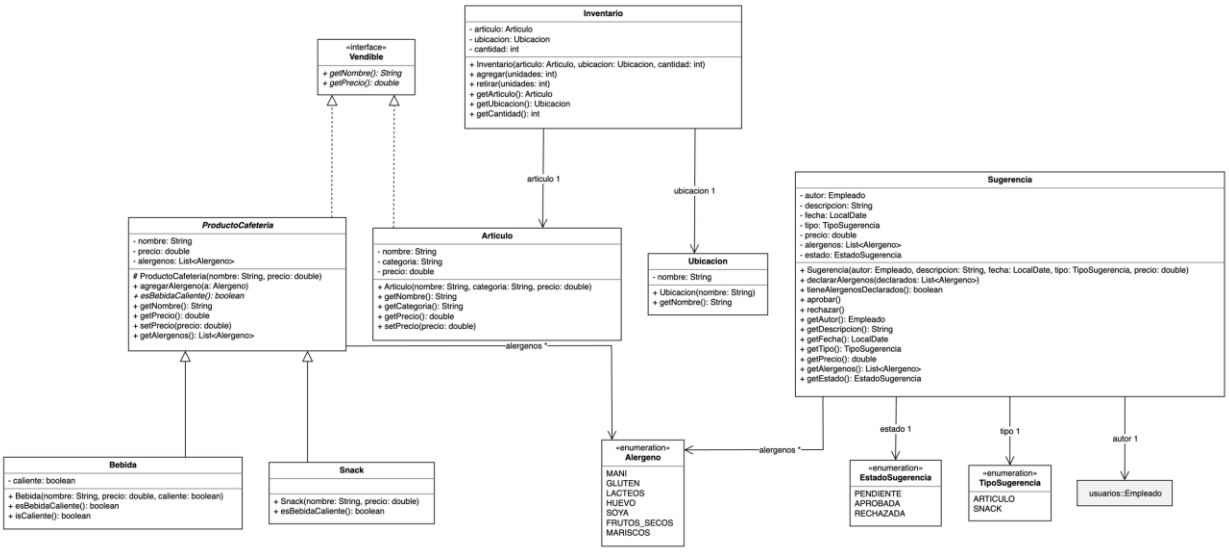
Competencia es abstracta y agrupa lo comun, Liga y CompetenciaPorFase heredan de ella pero cada uno implementa registrarResultado() a su manera, porque calcular el puntaje es muy distinto según el formato. NominaCompetencia conecta un equipo inscrito con sus jugadores habilitados, y PartidoCompetencia también reutiliza Partido.

Figura 7.7 Diagrama de clases detallado: Entrenamiento



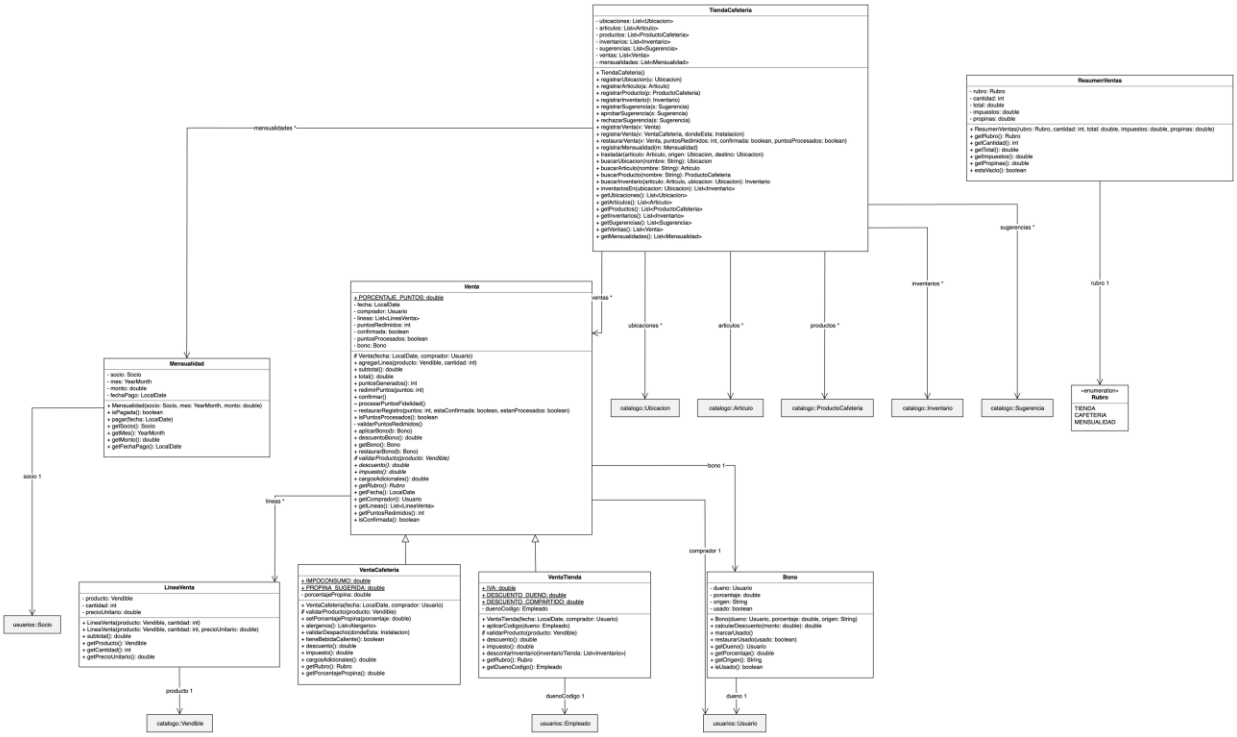
SesionEntrenamiento guarda por composición la lista de convocados, cada uno con su estado y opcionalmente una EvaluacionDesempeno con nota y comentarios. Separar la asistencia de la evaluación permite registrar que alguien asistió sin tener que calificarlo de inmediato.

Figura 7.8 Diagrama de clases detallado: Catálogo



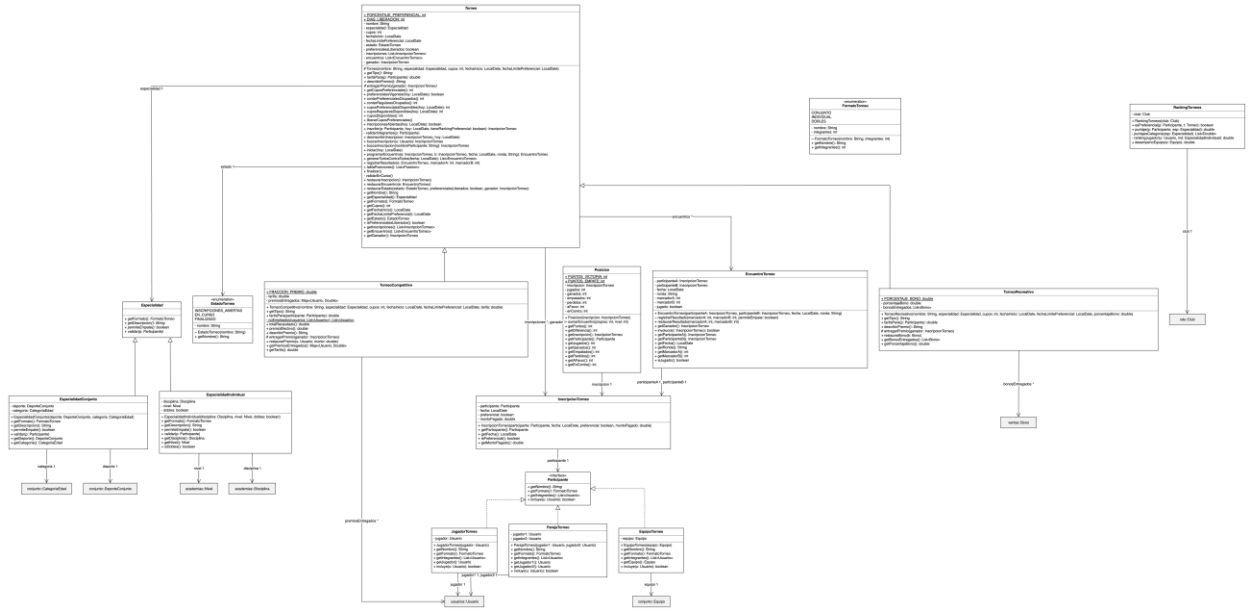
Vendible es una interfaz que exige solo nombre y precio, implementando por Articulo y por ProductoCafeteria, asi LineaVenta guarda cualquier tipo de producto sin conocer sus detalles. Ventas es abstracta y define el algoritmo general, mientras VentaTienda y VentaCafeteria implementan su propio descuento, impuesto y validación de que productos aceptan

Figura 7.9 Diagrama de clases detallado: Ventas



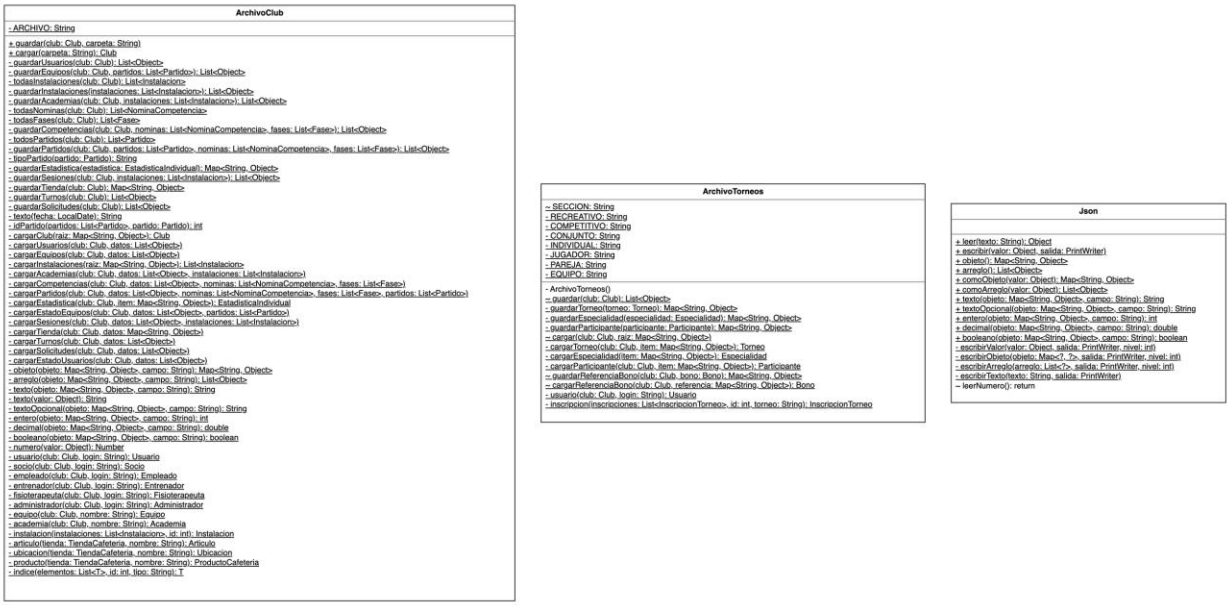
Venta guarda ahora un Bono opcional: aplicarBono() lo rechaza si ya se usó, si no es del comprador o si la venta ya tiene puntos o código de descuento, y confirmar() lo marca como usado. ResumenVentas reúne los totales de un rubro para el reporte de ingresos.

Figura 7.10 Diagrama de clases detallado: Torneos



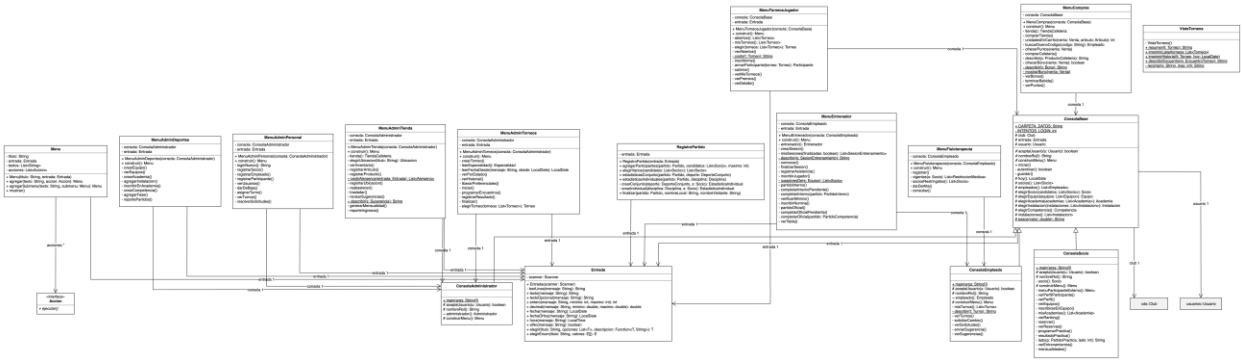
Torneo es abstracto y reúne los cupos, las inscripciones, los encuentros y la tabla de posiciones, TorneoRecreativo y TorneoCompetitivo solo cambian la tarifa y el premio con tarifaPara() y entregarPremio(). Especialidad separa qué se juega: EspecialidadConjunto (deporte y categoría) acepta solo equipos completos y EspecialidadIndividual (disciplina, nivel y si es de dobles) acepta jugadores o parejas. Participante es una interfaz que implementan JugadorTorneo, ParejaTorneo y EquipoTorneo, así que una inscripción no necesita saber qué tipo de participante tiene. RankingTorneos consulta los rankings de las academias y los puntos de las ligas para decidir los cupos preferenciales.

Figura 7.11 Diagrama de clases detallado: Persistencia



ArchivoClub guarda y carga toda la información del club en datos/club.json, apoyándose en Json para convertir entre texto y estructuras de Java. ArchivoTorneos se encarga de la sección de torneos del mismo archivo, incluidos los bonos y premios entregados.

Figura 7.12 Diagrama de clases detallado: Consola



ConsolaBase reúne lo común a las tres consolas: carga los datos, autentica, muestra el menú del rol y guarda al salir. ConsolaAdministrador, ConsolaSocio y ConsolaEmpleado tienen cada una su main y arman su menú con Menu, cuyas opciones son objetos Accion. Entrada concentra la lectura y validación de todas las entradas, y cada área tiene su propia clase de menú (MenuAdminTorneos, MenuCompras, MenuEntrenador, entre otras), así ninguna clase de la consola crece demasiado y agregar una opción es una sola línea.

8. Decisiones de diseño

Decisión 1: Interfaz Vendible en lugar de una clase padre Producto

Problema	Una venta debe poder tener artículos de la tienda y productos de la cafetería, pero esas dos cosas casi no se parecen: el artículo tiene categoría e inventario, y el producto de cafetería tiene alérgenos y puede ser una bebida caliente.
Alternativas	(a) Clase abstracta Producto, padre de Articulo y ProductoCafeteria. (b) Interfaz Vendible con getNombre() y getPrecio(). (c) Dos tipos de LineaVenta separados.
Decisión	(b) Interfaz Vendible. LineaVenta guarda un Vendible y cada tipo de venta valida cuál acepta en validarProducto().
Se gana	LineaVenta y Venta no dependen de los detalles de cada producto. Articulo y ProductoCafeteria pueden tener jerarquías propias sin forzar una herencia que no tienen en común. Agregar un nuevo tipo de producto vendible solo exige implementar dos métodos.
Se sacrifica	Al recibir un Vendible, la venta tiene que comprobar el tipo concreto (instanceof) para saber si lo acepta. No hay atributos compartidos, así que nombre y precio se repiten en las dos clases.
Relación	RF-S11 y RF-S12 (comprar en la tienda y en la cafetería) y RNF-02.

Decisión 2: Persistencia en JSON escrita a mano

Problema	El enunciado exige que toda la información persista en archivos, en una carpeta distinta al código, y el formato lo decide el grupo.
Alternativas	(a) Serialización binaria de Java (Serializable). (b) Una librería como Gson. (c) JSON leído y escrito con una clase propia (Json) y una clase de persistencia por módulo.
Decisión	(c) JSON con la clase propia Json y ArchivoClub para el módulo comercial.
Se gana	Los archivos se pueden abrir y leer, lo que ayuda a revisar los datos de prueba. No hay que agregar librerías externas al proyecto de Eclipse. Las clases del dominio no saben nada del formato (RNF-03): si mañana se cambia a una base de datos, solo cambia el paquete persistencia.
Se sacrifica	Más código que con Serializable o Gson, porque cada clase se convierte campo por campo en las dos direcciones. Cada vez que una clase cambia sus atributos, hay que actualizar su persistencia a mano.
Relación	RNF-01 (persistencia) y RNF-03 (independencia del almacenamiento).

Decisión 3: Las referencias se guardan por nombre o login

Problema	Los objetos se apuntan entre sí: una venta apunta a su comprador y a sus productos, y un inventario a su artículo y su ubicación. En un archivo no se puede guardar “una referencia a otro objeto”.
Alternativas	(a) Copiar el objeto completo dentro de cada uno que lo usa. (b) Asignar un número de identificación artificial a cada objeto. (c) Guardar el nombre (artículos, productos, ubicaciones) o el login (usuarios), y buscar el objeto al cargar.
Decisión	(c) Guardar por nombre o login. Al cargar, primero se leen los objetos que son referenciados (ubicaciones y artículos) y después los que los usan (inventarios y ventas).
Se gana	Cada objeto existe una sola vez en memoria después de cargar, así que las comparaciones por referencia del modelo siguen funcionando. No hay que inventar identificadores nuevos, porque el login ya es único (RF-S01). El archivo se entiende al leerlo: "comprador": "ana".
Se sacrifica	Los nombres de artículos, productos y ubicaciones tienen que ser únicos, TiendaCafeteria lo valida al registrar. El orden de carga importa, y los usuarios tienen que cargarse antes que la tienda.
Relación	RNF-01 y la regla de login único.

Decisión 4: Competencia como clase abstracta con Liga y CompetenciaPorFase como subclases, en vez de una sola clase

Problema	Una competencia puede organizarse como liga(todos contra todos, tabla de posiciones) o por fases (grupos + eliminaciones directas), y la forma de calcular puntaje/clasificación tras un resultado es completamente distinta en cada caso.
Alternativas	(a) Una sola clase Competencia con atributo tipoFormato y un if/else dentro de registrarResultado() para decidir que hacer segun el resultado. (b) Una clase abstracta

	Competencia con los datos y reglas comunes y un metodo abstracto regisrarResultado(), implementando por sparado en Liga y CompetenciaPorFase(). (c) Dos clases totalmente independientes sin relación entre ellas, repitiendo la logica de equipos en cada una.
Decision	(b) Competencia abstracta registrarResultado() como metodo abstracto, Liga y CompetenciaPorFase heredan todo lo comun y cada uno decide como calcular el resulado a su manera
Se gana	La logica compartida se escribe una sola vez, agregar un tercer formato de competencia en el futuro solo exigira crear otra subclase e implementar registrarResultado(), sin tocar las otras dos
Se sacrifica	Al tener registrarResultado() como contrato obligatorio, cualquier subclase nueva esta forzada a implementarlo, aunque no se sepa bien que debe hacer
Relacion	RF-EN06 y RF-EN07

Decisión 5: Especialidad separada del tipo de torneo

Problema	Un torneo es recreativo o competitivo y, por otro lado, de conjunto, individual o de dobles, combinar las dos cosas en subclases daría seis clases.
Alternativas	(a) Una subclase por cada combinación. (b) Atributos de tipo y modalidad con condicionales. (c) Herencia para el tipo de torneo y una clase Especialidad aparte para lo que se juega.
Decisión	(c) Torneo tiene una Especialidad (EspecialidadConjunto o EspecialidadIndividual), y TorneoRecreativo o TorneoCompetitivo definen la tarifa y el premio.
Se gana	Cada eje cambia por separado: un nuevo tipo de premio es una subclase de Torneo y una nueva forma de juego es una subclase de Especialidad, sin multiplicar clases.
Se sacrifica	Para conocer todo un torneo hay que mirar dos objetos, y Especialidad necesita conocer a los participantes para validarlos.

Relación	HU-ADM-04, HU-ADM-05 y HU-SOC-03.

Decisión 6: Participante como interfaz

Problema	En un torneo se inscriben jugadores, parejas o equipos, y la inscripción, los encuentros y la tabla deben tratarlos igual.
Alternativas	(a) Que InscripcionTorneo guarde un jugador, una pareja o un equipo en atributos distintos. (b) Una clase padre abstracta. (c) Una interfaz Participante.
Decisión	(c) Participante con getNombre(), getFormato(), getIntegrantes() e incluye(), implementada por JugadorTorneo, ParejaTorneo y EquipoTorneo.
Se gana	Torneo, InscripcionTorneo y Posicion no dependen del tipo de participante, las reglas que necesitan personas (turnos, premios y bonos) usan getIntegrantes().
Se sacrifica	Algunas reglas tienen que preguntar el tipo concreto, por ejemplo para validar que un equipo esté completo.
Relación	HU-SOC-03, HU-EMP-01 y HU-ADM-06.

Decisión 7: Consola con menú y entrada reutilizables

Problema	Hay tres consolas con muchas opciones y todas deben validar sus entradas, una sola clase por consola crecería a miles de líneas.
Alternativas	(a) Una clase por consola con un switch grande. (b) Una clase base, un Menu genérico con acciones y una clase Entrada para validar.
Decisión	(b) ConsolaBase con el flujo común, Menu y Accion para las opciones, Entrada para leer y validar, y una clase de menú por área.
Se gana	Agregar una opción es una línea (menu.agregar(texto, accion)), la validación está en un solo lugar y ninguna clase de la consola es muy larga.
Se sacrifica	Hay más clases pequeñas que recorrer para seguir un flujo completo.
Relación	RNF-02 y el requisito de una consola por tipo de usuario.

9. Riesgos y consistencia

9.1 Validación del diseño

Para validar el diseño revisamos las cuatro recomendaciones de la guía y cómo las cumple el sistema.

Reducir la complejidad: cada paquete trabaja un solo dominio del club (usuarios, conjunto, partidos, academias, competencias, entrenamiento, catálogo, ventas y torneos), por lo que ninguna clase hace todo. Club funciona como coordinador: recibe las solicitudes y las delega a la clase que tiene la información. Como, Equipo valida el cupo y el número de camiseta, Academia valida las reservas y Venta calcula su propio total.

Pensar en los cambios: diseñamos las partes que más pueden cambiar para que el cambio quede en un solo lugar. Si aparece un nuevo tipo de producto, basta con crear una clase que implemente Vendible, sin tocar Venta ni LineaVenta. Si se crea un nuevo formato de competencia, basta con otra subclase de Competencia. Si aparece un nuevo tipo de torneo, basta con otra subclase de Torneo que defina tarifaPara() y entregarPremio(). Y si en el futuro se migra de archivos JSON a una base de datos, solo cambian ArchivoClub y Json, lo que responde al RNF-03.

Reducir el acoplamiento: las clases conocen lo mínimo de las demás. Venta y LineaVenta solo conocen la interfaz Vendible y no las clases concretas como Articulo o Bebida. Las clases del dominio tampoco saben nada de archivos ni de la consola, así que la lógica no depende de cómo se muestra ni de cómo se guarda la información (RNF-02 y RNF-03).

Inyección de dependencias: se cumple de forma parcial. Clases como Reserva, Turno y SesionEntrenamiento reciben por constructor los objetos con los que trabajan, en lugar de buscarlos o crearlos ellas mismas, y ArchivoClub recibe el Club como parámetro. La excepción es Club, que crea su propia TiendaCafeteria, lo dejamos así porque el club solo tiene una tienda y no se necesita reemplazarla.

9.2 Preocupaciones transversales

Persistencia: toda la información del club se guarda en el archivo club.json, dentro de la carpeta de datos, que está separada del código. ArchivoClub es la única clase que conoce ese formato y usa la clase Json para convertir entre texto y objetos de Java. Como se explicó en la decisión 3, las referencias entre objetos se guardan por login o por nombre, así que al cargar el orden importa: primero se leen los usuarios y los equipos, y después lo que depende de ellos, como partidos, academias, ventas y turnos. Con esto se cumple el RNF-01, porque la información se conserva entre ejecuciones. Los torneos se guardan en el mismo archivo, en su propia sección, a cargo de ArchivoTorneos.

Errores: cuando una operación viola una regla del negocio, la clase que tiene la información lanza una ReglaNegocioException con un mensaje que explica el motivo, por ejemplo "Ya existe

un usuario con el login ana" o "No hay cupo". ProgramaClub atrapa esa excepción y también los errores de lectura del archivo (IOException), y muestra el mensaje sin que la aplicación se cierre. Esto ayuda al RNF-04, porque el resultado de cada prueba se ve de forma clara.

Validaciones: las reglas se validan en la clase que tiene los datos y no en la consola. Entre otras cosas, el sistema valida que no se repita un login, que una academia no supere su cupo de instalaciones en un mismo horario, que no se use una instalación no permitida con bebida caliente, que la hora de fin de una reserva sea posterior a la de inicio y que un empleado solo pueda pedir el cambio de su propio turno.

9.3 Cambios previstos

Pensamos en los cambios que tienen más probabilidad de pasar y en qué parte del sistema afectarían:

Si el club abre un nuevo deporte de conjunto, solo se agrega una constante en DeporteConjunto con su mínimo de jugadores. Si la tienda o la cafetería venden un nuevo tipo de producto, se crea una subclase de ProductoCafeteria o una clase que implemente Vendible. Si aparece una nueva forma de organizar competencias, se crea una subclase de Competencia que implemente registrarResultado(). Si se cambia la forma de guardar la información, por ejemplo, a una base de datos, solo se reemplaza ArchivoClub. Si la interfaz pasa de consola a menús o a una interfaz gráfica, solo cambia el paquete consola, porque la lógica no depende de ella. Si se agrega una nueva forma de premiar, se crea una subclase de Torneo, y si aparece una nueva forma de juego, una subclase de Especialidad. El 70% del premio, el 15% del bono y los 3 días de liberación de cupos son constantes, cada una en una sola clase.

9.4 Coherencia con el código

Para revisar la coherencia comparamos los diagramas con el código actual del proyecto. Los diagramas de clases detallados de la sección 7 y el modelo de dominio de la sección 6 se generaron a partir del código, por lo que coinciden los nombres de las clases, los atributos, los métodos con sus firmas y las relaciones. Los mensajes de los diagramas de secuencia corresponden a métodos que aparecen en el diagrama detallado. Además, cada historia de usuario tiene su prueba JUnit (sección 1.8), y las 136 pruebas del proyecto pasan.

Uso de IA

Durante el desarrollo de este trabajo se utilizó la herramienta de IA Claude (Anthropic) como apoyo en tres aspectos puntales. En primer lugar, se empleó para generar un diagrama de ejemplo que sirvió como referencia para la elaboración de los diagramas propios del proyecto, los cuales fueron diseñados y construidos por el equipo. En segundo lugar, se usó como apoyo en la revisión del código, con el fin de identificar posibles errores o fallas en su funcionamiento. Por

último, se utilizó para corregir aspectos gramaticales y mejorar la redacción de algunos apartados del documento.

En el Proyecto 2 se usó Claude Code (Anthropic) como apoyo para auditar el código frente al banco oficial de historias de usuario. Posteriormente, una vez ajustamos el código y creamos las nuevas clases y métodos para cumplir con el banco oficial de HU, planteamos las nuevas HU para Torneos y con base en ello le pedimos a la IA una lluvia de ideas para posibles escenarios de prueba JUnit. Con el equipo implementamos la pruebas y Claude nos ayudó a revisarlas para asegurarnos de que la implementación estuviera correcta. Por último, tras reajustar los diagramas UML, Claude nos ayudó a reorganizarlos y formatearlos para que se vieran más legibles.